Crypto Intelligence Agent

Relatório técnico das Fases 1 a 5

Gabriel Aguiar Aquino

8 de novembro de 2026

Sumário

Repositório do código **4**

Declaração de uso de inteligência artificial **5**

1 Introdução **6**
1.1 Princípio central . . . 6
1.2 Ambiente de execução . . . 6
1.3 Organização do relatório . . . 7

2 Arquitetura geral **8**
2.1 Visão de componentes . . . 8
2.2 Anatomia do agente . . . 8
2.3 Fluxo de um turno do agente integrado . . . 9
2.4 Armazenamento . . . 9
2.5 Pilha e versões finais . . . 9
2.6 Interface . . . 9

3 Fase 1 — Chain estruturada com LangChain e Ollama **11**
3.1 Objetivo . . . 11
3.2 Escopo . . . 11
3.3 Solução implementada . . . 11
3.4 Decisões técnicas . . . 12
3.5 Problemas encontrados e como foram resolvidos . . . 13
3.5.1 Import circular entre `app/chains` e `app/services` . . . 13
3.5.2 Aviso de permissão do cache do pytest no container . . . 13
3.5.3 Qualidade da saída do modelo — revisão iterativa do prompt . . . 14
3.5.4 O plano não respondia ao usuário — inclusão da resposta mínima . . . 15
3.5.5 Por que a resposta não pode conter valores de mercado . . . 17
3.5.6 Benchmark de modelos: qwen2.5:7b × gemma4:e4b . . . 17
3.6 Evidências e métricas . . . 19
3.7 Limitações conhecidas . . . 20
3.8 Aprendizados . . . 21
3.9 Pendências e insumos para a próxima fase . . . 21

4 Fase 2 — Agente com ferramentas, roteamento, memória e base vetorial **22**
4.1 Objetivo . . . 22
4.2 Escopo . . . 22
4.3 Solução implementada . . . 23
4.4 Decisões técnicas . . . 23

4.5 Problemas encontrados e como foram resolvidos . . . 24
4.5.1 Compatibilidade das APIs legadas . . . 24
4.5.2 Comportamento real da CoinPaprika . . . 24
4.5.3 Primeira execução real do agente . . . 24
4.5.4 Comparações erradas do modelo: três iterações . . . 24
4.5.5 Formato da aplicação: scripts → CLI única (descartada) → HTTP . 25
4.5.6 Exemplos do Swagger . . . 26
4.5.7 Retry de LLM . . . 26
4.5.8 Troca de FAISS por Chroma . . . 26
4.6 Evidências e métricas . . . 27
4.7 Limitações conhecidas . . . 27
4.8 Aprendizados . . . 27
4.9 Pendências e insumos para a próxima fase . . . 27

5 Fase 3 — Memória conversacional e raciocínio contextual 29
5.1 Objetivo . . . 29
5.2 Escopo e adaptações do enunciado . . . 29
5.3 Solução implementada . . . 29
5.4 Decisões técnicas . . . 30
5.5 Problemas encontrados e como foram resolvidos . . . 30
5.5.1 Escolha do framework de memória: três spikes . . . 30
5.5.2 30 dias da CoinPaprika . . . 31
5.5.3 Ajustes de regras e testes . . . 31
5.5.4 Problemas revelados pela execução real . . . 31
5.6 Evidências e métricas . . . 32
5.7 Limitações conhecidas . . . 32
5.8 Aprendizados . . . 32
5.9 Pendências e insumos para a próxima fase . . . 33

6 Fase 4 — Ferramentas externas e integração com o RAG existente 34
6.1 Objetivo . . . 34
6.2 Escopo . . . 34
6.3 Solução implementada . . . 35
6.4 Decisões técnicas . . . 35
6.5 Problemas encontrados e como foram resolvidos . . . 36
6.5.1 Primeira escolha de exemplo few-shot contaminava o experimento 36
6.5.2 Política de escolha entre busca original e reescrita (três versões) . 36
6.5.3 Erros semânticos do gerador SQL . . . 37
6.5.4 CTE recursiva negada pelo authorizer com erro genérico . . . 37
6.5.5 Agente real: pergunta combinada sem documento e entrada resumida . . . 37
6.5.6 Regressão completa: duas falhas reais . . . 38
6.5.7 Testes rodavam o código antigo da imagem . . . 38
6.6 Evidências e métricas . . . 38
6.7 Limitações conhecidas . . . 39
6.8 Aprendizados . . . 39
6.9 Pendências e insumos para a próxima fase . . . 40

7 Fase 5 — Agente integrado, verificação de respostas, avaliação e interface 41
7.1 Objetivo . . . 41
7.2 Escopo . . . 41
7.3 Solução implementada . . . 42

7.4 Decisões técnicas . . . 42
7.5 Problemas encontrados e como foram resolvidos . . . 43
7.5.1 Atualização do `langchain-core` exigida pelo `create_agent` . . . 43
7.5.2 NLI com modelo de 2B: neutro classificado como contradição . . . 44
7.5.3 Pergunta sobre o próprio histórico sem resposta . . . 44
7.5.4 Resposta cautelosa marcando a própria limitação . . . 44
7.5.5 Avaliação contaminada por falhas de infraestrutura . . . 45
7.5.6 Interface: retomar conversa pelo id . . . 45
7.5.7 Parâmetro depreciado na interface . . . 45
7.6 Evidências e métricas . . . 45
7.6.1 Avaliação final . . . 46
7.6.2 Integração final . . . 47
7.7 Limitações conhecidas . . . 47
7.8 Aprendizados . . . 47
7.9 Pendências . . . 48

8 Decisões transversais **49**
8.1 Política de não usar APIs depreciadas . . . 49
8.1.1 A diretriz . . . 49
8.1.2 O que foi trocado . . . 49
8.1.3 Como a regra foi verificada . . . 50
8.1.4 A exceção: APIs legadas exigidas na Fase 2 . . . 50
8.2 Desvios deliberados do enunciado . . . 51
8.3 Modelo local (Ollama) em CPU . . . 51
8.3.1 Por que um modelo local e pequeno . . . 51
8.3.2 Hardware . . . 52
8.3.3 Escolha do modelo: benchmark . . . 52
8.3.4 Resultado . . . 52
8.4 Código determinístico onde o modelo pequeno falha . . . 53
8.5 Segurança . . . 53
8.6 Estratégia de testes . . . 53

9 Conclusão **54**
9.1 O que foi entregue . . . 54
9.2 Resultados consolidados . . . 54
9.3 O que funcionou . . . 54
9.4 Limitações . . . 55
9.5 Trabalhos futuros . . . 55

Referências **56**

Repositório do código

Todo o código deste trabalho está no GitHub:

https://github.com/gabriel-q7/crypto-intelligent-agent

Diretório	Conteúdo
`app/`	Aplicação FastAPI: chains, agentes, ferramentas, memória, Text-to-Query, Retrieval Agent, verificação e avaliação
`ui/`	Interface Streamlit (só consome a API)
`tests/`	Testes offline (`tests/unit`, com LLM e API simulados) e de integração (`tests/integration`, com Ollama e CoinPaprika reais)
`knowledge/corpus/`	Corpus local de conceitos (sem preços) indexado no Chroma
`evaluation/`	Dataset versionado da avaliação da Fase 5
`benchmarks/`	Perguntas e resultados do benchmark de modelos da Fase 1
`docs/`	Documentação de referência do sistema e evidências brutas (`docs/evidence/`)
`Report/`	Diários técnicos por fase e capítulos comuns deste relatório
`scripts/`	Geração deste relatório e verificação automática dos números citados

Tudo roda localmente com Docker Compose (`make up`). O relatório pode ser regenerado com `make report` (ou `./scripts/generate-report.sh`). Antes de gerar o PDF, o script confere os números citados contra os arquivos de evidência versionados.

Declaração de uso de inteligência artificial

Este trabalho foi desenvolvido com o apoio de duas ferramentas de inteligência artificial generativa:

- **ChatGPT** (OpenAI), usado no **planejamento** das fases;
- **Claude Code** (Anthropic), usado como assistente de programação e de redação.

Como as ferramentas foram utilizadas

- **Planejamento (ChatGPT):** interpretação dos enunciados e organização das etapas de cada fase.
- **Implementação (Claude Code):** geração e revisão do código (chains, agentes, ferramentas, memória, Text-to-Query, Retrieval Agent, verificação, avaliação, interface e testes), a partir dos enunciados e das decisões do autor.
- **Investigação técnica (Claude Code):** spikes de bibliotecas (frameworks de memória, base vetorial, tool calling), checagem de avisos de depreciação e diagnóstico de falhas com o modelo e as APIs reais.
- **Relatório (Claude Code):** organização e redação dos diários de fase e deste relatório a partir das evidências registradas no repositório, além do script que consolida o relatório em PDF e confere os números.

Responsabilidades do autor

- Decisões de projeto, incluindo os desvios deliberados do enunciado (registrados no capítulo de decisões transversais).
- Execução do sistema, conferência das saídas e revisão crítica do código, das análises e do texto.
- Responsabilidade integral pelo conteúdo entregue.

Integridade dos resultados

Os números de desempenho, avaliação e testes citados vêm de execuções reais registradas no repositório (`benchmarks/results/`, `docs/evidence/`). As ferramentas de IA não produziram nem estimaram valores experimentais. As execuções com falha, inclusive as de infraestrutura, são relatadas, e não omitidas.

1 Introdução

Este relatório descreve a construção do **Crypto Intelligence Agent**, um assistente de análise de criptoativos que roda inteiramente na máquina local: o modelo de linguagem é servido pelo Ollama em CPU, e os dados vêm de fontes reais (API da CoinPaprika, um banco SQLite de snapshots e um corpus de documentos indexado no Chroma).

O sistema foi construído em cinco fases incrementais. Cada fase preservou as anteriores, que continuam acessíveis pela mesma API.

Fase	Tema	Entrega principal
1	Chain estruturada	Chain LCEL que transforma a pergunta num plano de análise em JSON validado, sem dados externos
2	Agente com ferramentas	Agente ReAct e chains roteadas com dados reais da CoinPaprika, base vetorial local e API HTTP
3	Memória conversacional	Sessões persistentes (SQLite), janela e resumo, memória episódica e semântica (Mem0 sobre Chroma)
4	Ferramentas externas e RAG	Banco de snapshots consultado por Text-to-Query com SQL validado; Retrieval Agent com Query Rewriting
5	Agente integrado e verificado	Agente com tool calling nativo, histórico persistente, verificação de respostas (NLI e coerência), avaliação no LangSmith e interface Streamlit

1.1 Princípio central

Nenhum valor de mercado é inventado. Esse requisito atravessa todas as fases e explica boa parte das decisões:

- na Fase 1, uma trava remove da resposta qualquer frase com valor monetário ou percentual, porque o modelo não tem dado atual;
- nas Fases 2 a 4, achados, fontes e horários são montados por código a partir das observações reais das ferramentas, e valores da resposta sem correspondência nos dados são removidos;
- na Fase 5, cada resposta é verificada contra a evidência da própria execução antes da entrega, com no máximo uma correção.

1.2 Ambiente de execução

Todas as medições deste relatório foram feitas na mesma máquina: um notebook com AMD Ryzen 5 5500U (6 núcleos, 12 threads) e 22 GB de RAM, Linux (Manjaro), Docker Compose e inferência só em **CPU**, sem GPU dedicada. O uso de um modelo local e pequeno é deliberado: os dados que importam vêm das ferramentas, e não da memória do modelo (ver o capítulo de decisões transversais). O modelo padrão é o `gemma4:e2b` com raciocínio interno (*thinking*) desligado, escolhido pelo benchmark da Fase 1. Os embeddings usam `nomic-embed-text`. A aplicação roda em Python 3.12 dentro de containers.

Executar em CPU tem consequências que aparecem em todo o relatório: um turno do agente leva de dezenas de segundos a alguns minutos, e várias decisões (checa-

gens determinísticas no lugar de chamadas ao LLM, reescrita de consulta só quando necessária) existem para conter essa latência.

1.3 Organização do relatório

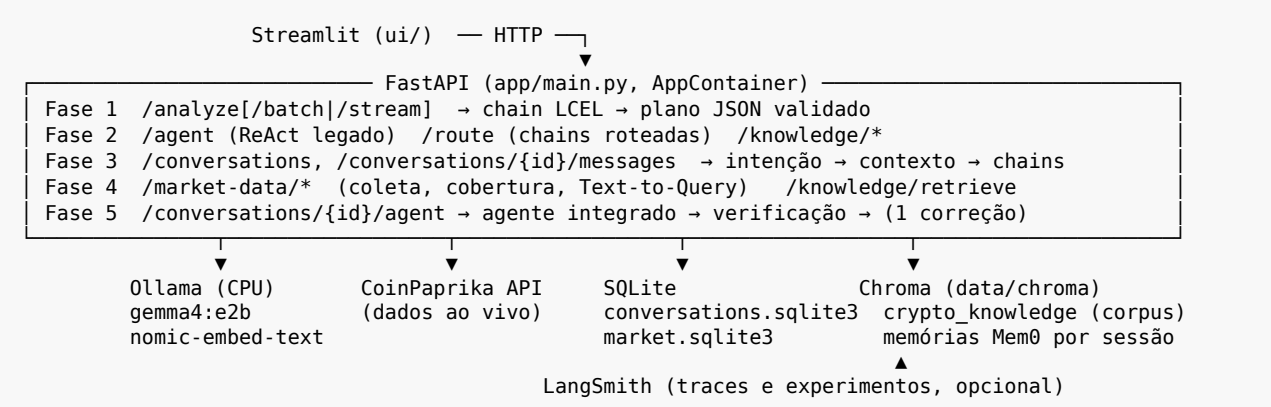
- **Arquitetura geral:** o sistema completo ao fim da Fase 5.
- **Fases 1 a 5:** um capítulo por fase, escrito como diário técnico durante o desenvolvimento (objetivo, escopo, solução, decisões, problemas, evidências, limitações, aprendizados e pendências). O capítulo da Fase 5 foi consolidado no fim do projeto a partir da documentação e das evidências da fase.
- **Decisões transversais:** as decisões que valem para o projeto todo, com destaque para a política de **não usar APIs depreciadas** e os desvios deliberados do enunciado.
- **Conclusão:** resultados consolidados, limitações e trabalhos futuros.

Os números citados vêm dos arquivos de evidência versionados (`benchmarks/results/` e `docs/evidence/`). Os diários usam datas absolutas, e as métricas trazem o modelo e o ambiente em que foram medidas.

2 Arquitetura geral

Este capítulo descreve o sistema como ele está ao fim da Fase 5. Os capítulos das fases mostram como ele chegou até aqui.

2.1 Visão de componentes



Os serviços do Docker Compose são `ollama`, `ollama-pull` (baixa os modelos uma vez), `app` (API) e `ui` (Streamlit). Os dados persistem em `./data` no host.

2.2 Anatomia do agente

Bloco	Fase 1	Fase 2	Fase 3	Fase 4	Fase 5
Percepção	`CryptoQuestion` (Pydantic)	Validação dos argumentos e extração de ativos	Intenção e entidades no contexto da conversa	Query Rewriting no vocabulário dos documentos	Argumentos tipados das ferramentas (tool calling)
Raciocínio	Prompt + `ChatOllama` → plano	ReAct e roteador com prompt PTCF	Resolução de continuação, correção e resultado anterior	O ReAct escolhe também banco, retrieval e cálculo; LLM gera SQL validado	`create_agent` com tool calling nativo; verificação e correção
Ação	—	CoinPaprika e busca semântica	Mesmas ferramentas, com contexto	SQL somente leitura, Retrieval Agent, cálculo	As mesmas ferramentas, combinadas numa execução
Memória	—	Janela limitada em processo	SQLite (janela + resumo) e Mem0 (episódica e semântica)	Sem mudança	Histórico persistente da Fase 3 lido por um adaptador

2.3 Fluxo de um turno do agente integrado

```
POST /conversations/{id}/agent
 └─ trava por sessão → grava a pergunta (SQLite)
    → SessionHistoryAdapter: resumo + memórias Mem0 + janela (só leitura)
    → create_agent: LLM ⇄ ferramentas (API, SQL, RAG, cálculo); trajetória registrada
    → VerificationService: afirmações → evidência → checagem numérica / NLI → coerência lógica → status
    → [se necessário] 1 correção com as próprias observações → nova verificação → resposta cautelosa se
↪  falhar
    → grava a resposta, a análise e o episódio → manutenção em segundo plano (resumo, título, extração)
```

2.4 Armazenamento

Repositório	Conteúdo	Papel
`data/conversations.sqlite3`	Sessões, mensagens, resumos, análises, contexto ativo	Fonte da verdade da conversa (Fase 3)
`data/market.sqlite3`	Snapshots reais da CoinPaprika	Dado de mercado histórico, consultado só por SQL validado (Fase 4)
Chroma `crypto_knowledge`	Trechos do corpus local com metadados	Base de conhecimento (Fases 2 e 4)
Chroma (Mem0)	Memórias episódicas e semânticas por sessão	Memória de longo prazo (Fase 3)

Os dois bancos SQLite são separados por decisão da Fase 4: apagar ou recriar um não afeta o outro.

2.5 Pilha e versões finais

Pacote	Versão	Uso
`langchain-core`	1.7.6	Runnables, prompts, mensagens (era 1.6.12 até a Fase 4)
`langchain`	1.4.9	`create_agent` (Fase 5)
`langchain-ollama`	1.1.0	`ChatOllama` e `OllamaEmbeddings`
`langchain-classic`	1.0.8	Só as APIs legadas exigidas na Fase 2, isoladas em `app/legacy/`
`langchain-chroma` / `chromadb`	1.1.0 / 1.5.9	Base vetorial
`mem0ai`	2.2.1	Memória de longo prazo
`sqlglot`	30.27.0	Validação do SQL gerado pelo LLM
`fastapi` / `uvicorn`	0.150.0 / 0.54.0	API HTTP
`pydantic`	2.14.1	Schemas e validação
`langsmith`	0.14.17	Rastreamento e avaliação (dependência do `langchain-core`)
`streamlit`	1.65.0	Interface

2.6 Interface

A interface Streamlit (`ui/`) só fala com a API por HTTP (`ui/backend.py`). Nenhuma lógica de agente, memória ou verificação roda nela. A barra lateral lista as conversas

com o título gerado pelo modelo após o primeiro turno. Cada resposta mostra o status da verificação, as afirmações sustentadas, neutras e contraditas, as ferramentas usadas e as fontes com horário.

3 Fase 1 — Chain estruturada com LangChain e Ollama

Período	2026-10-17
Status	Concluída
Ambiente	Linux (Manjaro), Docker Compose 5.5.1, inferência em **CPU** (12 threads, sem GPU)
Modelo	`qwen2.5:7b` via `ollama/ollama:latest` (4,7 GB)
Versões	Python 3.12.15 (container), `langchain-core 1.6.12`, `langchain-ollama 1.1.0`, `pydantic 2.14.1`, `pytest 9.1.1`

3.1 Objetivo

Construir uma chain de ponta a ponta que **interprete** perguntas sobre criptomoedas e produza um JSON estruturado e validado, **sem consultar APIs externas**.

O sistema não responde a pergunta do usuário. Ele produz um **plano de análise**: intenção, ativos, métricas necessárias, período, etapas de investigação, lacunas, premissas e confiança. Esse plano é o contrato de entrada das fases seguintes, quando ferramentas e APIs passarão a executar as etapas.

Critérios de conclusão:

- análise executada pela CLI, retornando JSON validado;
- mesma chain em duas composições (LCEL e `@chain`);
- demonstração de `invoke`, `batch` e `stream`;
- execução local com Docker Compose.

Todos foram atendidos (seção 6).

3.2 Escopo

Dentro: schemas Pydantic, prompts, `ChatOllama`, `JsonOutputParser` com validação, duas composições, interface Runnable, Docker Compose, testes unitários e de integração, documentação.

Fora, por decisão de fase: integrações com CoinPaprika, DeFiLlama e CoinGecko; memória persistente; RAG; ferramentas; planejamento autônomo. O objetivo era isolar e medir a qualidade da **interpretação** antes de acoplar dados reais.

3.3 Solução implementada

```
CryptoQuestion ⟶ build_prompt_input ⟶ ANALYSIS_PROMPT ⟶ ChatOllama ⟶ JsonOutputParser ⟶
↪  validate_analysis ⟶ CryptoAnalysisRequest
 (Pydantic)        (PromptTemplate +      (ChatPromptTemplate:   (format=json,   (texto → dict,
↪  (dict → modelo,
                   data de referência)    system + few-shot      temperature=0)  parciais no stream)
↪  erros de domínio)
                                          + pergunta)
```

Camada	Arquivo	Responsabilidade
Schemas	app/schemas/	`CryptoQuestion` (entrada) e `CryptoAnalysisRequest` (saída), com normalização de tickers e enum de intenção
Prompts	app/prompts/analysis.py	Instruções de sistema, `PromptTemplate` da pergunta, exemplo few-shot, vocabulário de métricas
Chains	app/chains/analysis.py	`build_lcel_chain` (padrão), `build_imperative_chain` (`@chain`), `build_json_chain` (streaming)
Serviços	app/services/	Fábrica do `ChatOllama`, validação final, `run_analysis` / `run_batch` / `stream_analysis` com tradução de erros
Erros	app/errors.py	`AnalysisOutputError` (saída inválida, guarda `raw_output`) e `LLMUnavailableError` (Ollama inacessível)
Entrada	scripts/	`analyze.py` (CLI) e `demo_runnables.py` (invoke, batch, stream)
Infra	compose.yaml, Dockerfile	Serviços `ollama`, `ollama-pull` (baixa o modelo uma vez) e `app`; volume `ollama_models`

3.4 Decisões técnicas

Decisão	Alternativas consideradas	Motivo
LCEL como composição padrão	`@chain` (imperativa)	A chain é linear. Só a LCEL propaga streaming etapa a etapa (o `@chain` emite um único chunk no fim), e ela permite reaproveitar subpartes (`build_json_chain`) e usar `batch`/`with_retry`/`with_fallbacks` em cada etapa. Detalhes em docs/composition-decision.md.

Decisão	Alternativas consideradas	Motivo
`format="json"` no `ChatOllama` + `temperature=0`	Só instrução no prompt	Restringe a geração a JSON sintaticamente válido e reduz a variação entre execuções. Não garante aderência ao schema, por isso existe a validação Pydantic.
`JsonOutputParser` + validação Pydantic separada	`with_structured_output` / `PydanticOutputParser`	O `JsonOutputParser` emite objetos parciais durante o streaming (requisito da fase); a validação estrita fica isolada em um passo próprio, testável sem modelo.
Erros de domínio (`AnalysisOutputError`, `LLMUnavailableError`)	Propagar exceções do LangChain/httpx	O chamador não precisa conhecer exceções de infraestrutura; `raw_output` preserva a saída do modelo para diagnóstico.
`batch` com `return_exceptions=True`	Falhar o lote inteiro	Uma resposta inválida do modelo não descarta as outras.
Serviço `ollama-pull` no Compose	Baixar o modelo manualmente	Subida reprodutível com um comando; o volume persistente evita novo download.
`requirements.txt` único (inclui pytest)	Separar dependências de dev	Menos arquivos; os testes rodam no mesmo container da aplicação.
Validação toda em container Python 3.12	venv local	O host tem apenas Python 3.14; o requisito é 3.12.

3.5 Problemas encontrados e como foram resolvidos

3.5.1 Import circular entre `app/chains` e `app/services`

- **Sintoma (identificado na revisão do código, antes da execução):** `app/chains` importa `app.services.validation`; se `app/services/__init__.py` reexportar `app.services.analysis`, que importa `app.chains`, o pacote é importado parcialmente.
- **Correção:** `app/services/__init__.py` sem reexports; quem usa importa o módulo diretamente.

3.5.2 Aviso de permissão do cache do pytest no container

- **Sintoma:** `PytestCacheWarning: ... Permission denied: '/app/...'`.
- **Causa:** a aplicação roda como usuário sem privilégios (`appuser`), mas `/app` pertencia ao root.
- **Correção:** `COPY --chown=appuser:appuser` e `chown` do diretório no Dockerfile.

3.5.3 Qualidade da saída do modelo — revisão iterativa do prompt

Problema principal da fase. A chain funcionava (JSON válido, schema respeitado), mas o **conteúdo** do plano tinha falhas. Pergunta de referência: `"Compare BTC e ETH" --assets BTC ETH --period "últimos 30 dias"`.

Saída inicial:

```
{
  "requested_metrics": ["rendimento", "volatilidade"],
  "investigation_steps": ["calcular o rendimento médio dos últimos 30 dias", "calcular a
  ↪  volatilidade...", "comparar..."],
  "missing_information": [],
  "assumptions": ["moeda de cotação USD"],
  "confidence": 1.0
}
```

Diagnóstico:

Campo	Falha	Causa raiz
`investigation_steps`	Os cálculos começavam sem nenhum passo para obter os dados	Nenhuma regra exigia uma etapa de coleta
`requested_metrics`	Termos vagos e instáveis entre execuções (“rendimento”, “preço_atual”) e sem o dado de base	Texto livre, sem vocabulário
`missing_information`	Dado de mercado aparecia às vezes aqui, às vezes em `requested_metrics`	Regra ambígua no prompt (“liste em requested_metrics e, se faltar, em missing_information”)
`assumptions`	Critério de comparação inferido e não registrado	Nenhuma regra pedia o registro de critérios inferidos
`confidence`	1.0 com critério inferido	Sem régua de calibração

Cada iteração foi validada com 5 perguntas reais: comparação, preço atual/previsão, TVL do Aave, staking de ETH e risco de SOL.

#	Mudança	Efeito na pergunta de referência	Efeito colateral
1	Regras novas: separação `requested_metrics` × `missing_information`, coleta como primeira etapa, régua de confiança, vocabulário `METRIC_VOCABULARY`	Etapa de coleta presente, critério registrado, confiança 0.8	`price` em vez de `price_history`; moeda/granularidade ausentes

#	Mudança	Efeito na pergunta de referência	Efeito colateral
2	Exemplo few-shot de **risco** (SOL × AVAX, sem período)	`price_history` correto	**Regressão:** `assumptions` vazio e `confidence` 1.0 de novo
3	Exemplo few-shot de **comparação** com período informado (padrão mais comum)	Resolvido: 3 premissas, confiança 0.8	`assets: []` em perguntas que citavam BTC/ETH só no texto
4	No exemplo, ativos citados só no texto (campo "Ativos informados" vazio)	Estável	Modelo "corrigiu" o período: *"período de 2026 é um erro, deve ser 2023"*
5	**Data de referência** injetada no prompt e regra para nunca alterar o período do usuário	Estável (3 execuções idênticas)	Ver limitações (seção 7)

Diagnóstico das iterações 2 a 4:

- **Iteração 2:** o modelo segue mais o padrão do exemplo do que as regras abstratas. Um exemplo com outro perfil de pergunta (sem período) não ensinou o caso com período explícito.
- **Iteração 3:** o modelo aprendeu a **copiar** os ativos do campo estruturado do exemplo, em vez de extraí-los do texto.
- **Iteração 4:** sem saber a data atual, o modelo assume a data de corte do treino e trata 2026 como erro. É um risco direto ao requisito de "não inventar dados".

Saída final para a pergunta de referência:

```
{
  "intent": "comparison",
  "assets": ["BTC", "ETH"],
  "requested_metrics": ["price_history", "return_pct", "volatility", "max_drawdown"],
  "period": "últimos 30 dias",
  "investigation_steps": [
    "Obter price_history diário de BTC e ETH em USD nos últimos 30 dias",
    "Calcular return_pct, volatility e max_drawdown de cada ativo",
    "Comparar desempenho e risco entre BTC e ETH"
  ],
  "missing_information": [],
  "assumptions": [
    "comparação por retorno e risco, pois o critério não foi informado",
    "cotação em USD",
    "preços de fechamento diários"
  ],
  "confidence": 0.8
}
```

3.5.4 O plano não respondia ao usuário — inclusão da resposta mínima

- **Sintoma (apontado na revisão do usuário):** "O que é staking de ETH?" retornava só o plano (`"Definir o que é staking de ETH"`, `"Explorar o processo..."`),

sem nenhuma explicação. A pergunta é conceitual e o modelo sabe respondê-la sem dados externos.

- **Causa:** o schema da especificação só tinha campos de **plano**; o modelo não tinha onde responder.
- **Correção:** dois campos novos no `CryptoAnalysisRequest`:
  - `answer_status`: `answered` (conceito respondido com conhecimento do modelo), `requires_market_data` (depende de dados de mercado), `needs_clarification` (pergunta ambígua);
  - `answer`: resposta de 2 a 4 frases.
  - `investigation_steps` passou a poder ser vazio quando `answered`.
  - Segundo exemplo few-shot, conceitual ("Como funciona o halving do Bitcoin?"), mostrando que fatos de protocolo (210.000 blocos, 21 milhões de BTC) são permitidos e preços não.

Iterações com o modelo real (6 perguntas: staking, DEX, comparação, preço/previsão, TVL, risco):

#	Mudança	Resultado	Problema revelado
1	Campos `answer_status`/`answer` no fim do JSON + exemplo conceitual	Staking e DEX respondidos	"BTC preço" e "Aave TVL" **falharam**: `investigation_steps` vazio com `requires_market_data`; "risco de SOL" classificado como conceito
2	`answer_status` movido para logo após `intent`; `answered` restrito a conceitos ("NÃO use para avaliar desempenho ou risco de um ativo específico")	Aave corrigido	"BTC preço" ainda sem etapas; SOL ainda `answered`
3	Etapa de coleta derivada de `requested_metrics` quando o modelo a omite	"BTC preço" corrigido	—

Diagnóstico da iteração 1: modelos autorregressivos geram o JSON **na ordem dos campos**. Com `answer_status` no fim, o modelo escrevia `investigation_steps: []` — imitando o exemplo conceitual — antes de decidir que tipo de resposta daria. Colocar a decisão antes das consequências corrigiu o caso do Aave sem nenhuma regra nova.

Diagnóstico da iteração 3: a saída crua do modelo para "BTC vai subir amanhã? Qual o preço atual?" estava correta em tudo (`requested_metrics: ["price"]`, resposta "Não consigo prever [...] não tenho dados de mercado atualizados"), exceto pelas etapas omitidas. Rejeitar a análise inteira por uma omissão derivável era desproporcional; a etapa "Obter price de BTC" é gerada pelo código e registrada em `assumptions`, sem inventar nada.

3.5.5 Por que a resposta não pode conter valores de mercado

Com um campo de texto livre, surgiu o risco de o modelo escrever números de mercado ("o staking rende 4% ao ano", "o BTC custa US$ 65.000"). Antes, o plano só citava **nomes** de métricas (`price_history`), nunca valores.

Motivos para bloquear:

1. **O modelo não tem dado atual.** O `qwen2.5:7b` conhece apenas o que viu até o fim do treinamento (2024). Preço, volume, TVL e rendimento mudam diariamente; um número do modelo estaria defasado em mais de um ano e seria lido pelo usuário como atual, já que não vem com data.
2. **Mesmo o dado antigo não é confiável.** Modelos pequenos não memorizam números com precisão: geram valores *plausíveis*. Sem fonte externa nesta fase, não há como verificar. O caso de "2026 é um erro, deve ser 2023" (5.3) já mostrou o modelo agindo a partir de uma visão desatualizada do mundo.
3. **Requisito da especificação:** "impedir a invenção de preços, volumes e dados atuais" e "não inventar dados de mercado".
4. **Prompt não é garantia.** Modelos 7B nem sempre obedecem instruções (5.3). A regra precisava de uma verificação determinística, assim como o formato tem o Pydantic.

Decisão do usuário sobre o comportamento da trava, entre três opções (rejeitar a análise, substituir a resposta, remover a trava): **substituir**. Implementação refinada: apenas as frases com valor monetário ou percentual são removidas e um aviso padrão é acrescentado ("Nesta fase não tenho acesso a dados de mercado atualizados e respondo apenas com o conhecimento do modelo [...]"); o ocorrido é registrado em `assumptions`. Assim a parte conceitual da resposta é preservada.

Complemento no prompt, também pedido pelo usuário: quando a pergunta depende de dados, a resposta deve **dizer explicitamente** que o agente não tem dados atualizados nesta fase, em vez de apenas descrever o que será analisado.

Limites conhecidos da trava (heurística por expressão regular, não garantia): falso positivo em termos como "ataque de 51%"; falso negativo em valores sem símbolo de moeda ("65 mil", "65k").

3.5.6 Benchmark de modelos: qwen2.5:7b × gemma4:e4b

Objetivo (pedido do usuário): medir o desempenho do modelo atual e comparar com o `gemma4:e4b`, com troca de modelo simples pelo env.

Troca de modelo: antes, `OLLAMA_MODEL` definido no shell não chegava ao container (o `env_file` não é sobrescrito pelo shell). O `compose.yaml` passou a interpolar `OLLAMA_MODEL` no serviço `app`, com prioridade shell > `.env` > padrão; como o `app` depende do `ollama-pull`, que usa a mesma variável, `OLLAMA_MODEL=gemma4:e4b docker compose run --rm app ...` baixa e usa o modelo novo. A CLI também ganhou `--model`.

Ferramenta: `scripts/benchmark.py` executa prompt → LLM → parser → validação separadamente para capturar as métricas do Ollama (`load_duration`, `prompt_eval_*`, `eval_*`), que a chain completa descarta, e verifica `answer_status`/`assets` contra o esperado em `benchmarks/questions.json`. `scripts/compare_benchmarks.py` gera a tabela comparativa.

Problemas de medição encontrados e corrigidos:

Sintoma	Causa	Correção
“Prompt tok/s” de 609 e 1.904 (a medição isolada dava 23 e 38)	O aquecimento usava a mesma pergunta do 1º caso; a execução seguinte reaproveitou o prompt inteiro do cache do Ollama (0,3 s de leitura) e a média de razões amplificou esse ponto	Aquecimento com pergunta fora do conjunto; taxas calculadas como soma dos tokens ÷ soma dos tempos
Gemma gerando 1.232 tokens para um JSON de ~250	`gemma4` tem **thinking** (raciocínio interno) ligado por padrão; os tokens contam no tempo mas não aparecem na saída	Nova variável `OLLAMA_REASONING` (vazio/true/false), opção `--reasoning` no benchmark e coluna “Thinking” na comparação
Benchmark completo estimado em ~1h40	3 repetições × 6 perguntas × 3 configurações em CPU	Com temperatura 0 as repetições do Qwen variaram < 2%; padrão reduzido para 1 repetição (`--runs` continua disponível)

Resultado:

Modelo	Thinking	Cold start (s)	Latência média (s)	Mediana (s)	p95 (s)	Tokens prompt	Prompt tok/s	Tokens saída	Saída tok/s	JSON válido	answer_status ok	assets ok
qwen2.5:7b	padrão (não suporta)	7.0	71.0	65.8	90.5	2514	66.1	153	4.6	100%	83%	100%
gemma4:e4b	desligado	8.2	83.1	83.9	87.3	2383	38.9	211	9.6	100%	**100%**	100%
gemma4:e4b	ligado	7.5	118.1	110.1	187.7	2385	38.8	554	9.8	100%	**100%**	100%
mistral:7b	padrão (não suporta)	16.5	141.9	140.8	163.0	2985	43.6	179	2.4	**67%**	100%[1]	100%[1]
gemma4:e2b	desligado	8.6	**42.3**	**43.2**	**46.8**	2383	**75.4**	178	**16.7**	100%	**100%**	100%

[1] Calculado só sobre as respostas válidas (4 de 6).

mistral:7b (adicionado depois, 1 repetição): o mais lento (geração 2,4 tok/s; o tokenizador gera ~2.985 tokens para o mesmo prompt, +19% que o Qwen) e o único com saídas inválidas — omitiu `confidence` em “Compare BTC e ETH” e `answer` em “TVL do Aave”. O modo JSON do Ollama garante sintaxe, não a presença dos campos; a validação Pydantic rejeitou as duas saídas, como esperado. Nas respostas válidas, conteúdo correto e aviso de ausência de dados presente. Dados: `benchmarks/results/20261017T235946_mistral-7b.json`.

gemma4:e2b (adicionado depois, thinking desligado, 1 repetição): o mais rápido em todas as métricas de tempo — latência média 42 s (−40% vs Qwen, −49% vs Gemma E4B), leitura do prompt 75 tok/s e geração 16,7 tok/s — com 100% de JSON válido, `answer_status` e `assets`. Respostas conceituais corretas (staking citando Proof-of-Stake) e aviso de ausência de dados em todas as perguntas de mercado; todas as etapas vieram do modelo (sem etapa derivada nem trava acionada). Ponto fraco:

planos mais rasos — 1 etapa no TVL do Aave e `max_drawdown` ausente na comparação. Dados: `benchmarks/results/20261018T002057_gemma4-e2b_thinking-off.json`.

Leitura dos resultados:

- O **tempo é dominado pela leitura do prompt** (~2.400 tokens): ~54% do total no Qwen e ~73% no Gemma. Encurtar o prompt é a alavanca de desempenho mais forte para qualquer modelo.
- O Gemma **gera 2× mais rápido** mas **lê o prompt mais devagar**; no total, sem thinking, é 17% mais lento que o Qwen e mais estável (p95 87 s × 91 s).
- O **thinking não compensa** neste caso de uso: +42% de latência média, p95 2× maior, mesma acurácia.
- Em **qualidade**, o Gemma acertou 6/6 classificações e não repetiu o erro conceitual do Qwen ("nós de mineração").

Recomendação (atualizada após o E2B): `gemma4:e2b` com `OLLAMA_REASONING=false` — o mais rápido (42 s por pergunta, −40% vs Qwen) e com acurácia máxima neste conjunto; ocupa 4,6 GB. A ressalva são planos mais rasos que os do E4B, o que importa a partir da fase em que as etapas forem executadas por ferramentas. `mistral:7b` descartado (mais lento e 33% de saídas inválidas); `gemma4:e4b` sem thinking fica como alternativa de plano mais detalhado; thinking não compensa. O padrão do projeto não foi alterado; a decisão fica para o usuário.

Decisão (2026-10-18, pelo usuário): o padrão do projeto passou a ser **`gemma4:e2b` com `OLLAMA_REASONING=false`** (`.env`, `.env.example`, `compose.yaml` e `app/config.py`). A semântica de `OLLAMA_REASONING` ficou: ausente → `false` (padrão do projeto); vazio → padrão do modelo; `true`/`false` explícitos. No Compose isso exigiu `${OLLAMA_REASONING-false}` (sem `:`), porque `:-` trataria o valor vazio como ausente. O `qwen2.5:7b` continua no volume como alternativa.

Ambiente após os testes: por falta de disco (o `/` chegou a 100%, 7,9 MB livres), a pedido do usuário só o `qwen2.5:7b` foi mantido no volume do Ollama; `gemma4:e4b`, `gemma4:e2b` e `mistral:7b` foram removidos após o benchmark (resultados preservados em `benchmarks/results/`; para repetir: `--pull`). Também foi corrigido um bug do `--pull`: o download usava o cliente com timeout de inferência (300 s) e falhava em modelos de alguns GB.

Limitações do benchmark: 6 perguntas e 1 repetição por modo do Gemma; acurácia medida só em `answer_status` e `assets` (correção do conteúdo de `answer` foi avaliada manualmente); apenas CPU.

3.6 Evidências e métricas

Detalhes completos e saídas brutas: docs/stage-1-evidence.md.

Verificação	Comando	Resultado
Testes unitários (LLM falso, sem modelo)	`docker compose run --rm --no-deps app pytest`	**63 passed** em ~0,6 s
Testes de integração (Ollama + `qwen2.5:7b`)	`docker compose run --rm app pytest -m integration`	**4 passed**: 173,9 s com o prompt inicial; 255,5 s com o prompt final
CLI fim a fim	`docker compose run --rm app`	JSON validado; ~110 s na 1ª chamada (inclui carga do modelo)

Verificação	Comando	Resultado
`invoke` LCEL × `@chain`	`python -m scripts.demo_runnables invoke`	Saídas **idênticas** para a mesma pergunta
`batch` (3 perguntas, concorrência 2)	`python -m scripts.demo_runnables batch`	3/3 válidas
`stream`	`python -m scripts.demo_runnables stream`	53 objetos parciais; o objeto final validado é igual ao do `invoke`
Não inventar dados	"BTC vai subir amanhã? Qual o preço atual?"	Nenhum número de mercado; o preço vira métrica a buscar

Cobertura dos testes unitários: schemas (normalização, obrigatórios, limites), prompts (tipos, regras anti-invenção, escape do JSON schema, few-shot válido, data de referência), parsing e tradução de erros, **as duas composições com o mesmo conjunto de testes**, equivalência LCEL × `@chain`, streaming parcial, isolamento de falhas no `batch`, configuração por variáveis de ambiente e `LLMUnavailableError` com o Ollama inacessível.

Custo do few-shot: a integração passou de 174 s para 256 s (+47%) por causa do prompt mais longo, inferido em CPU.

Após a resposta mínima (5.4/5.5): 80 testes unitários (0,7 s) e 5 de integração (409,8 s, incluindo o novo teste "pergunta conceitual é respondida"). O tempo maior vem do segundo exemplo few-shot e do texto adicional gerado em `answer`.

3.7 Limitações conhecidas

1. **O streaming só existe antes da validação.** O Pydantic precisa do objeto completo; os objetos parciais não são validados e as chains com validação emitem um único chunk.
2. **Variação do modelo 7B após a revisão do prompt:**
   - `assets` oscila em perguntas conceituais ("O que é staking de ETH?": `[]` em uma execução, `["ETH"]` em outra);
   - a ressalva de que o agente não faz previsões nem sempre aparece;
   - `confidence` se concentra em 0.6 e 0.8; é autoavaliação, não uma medida calibrada.
3. **A não invenção de dados é garantida só pelo prompt.** Nada verifica automaticamente o texto livre (ex.: `investigation_steps`).
4. **Desempenho em CPU:** 30–110 s por chamada.
5. **Enum de intenção estrito:** uma intenção fora da lista é rejeitada; não há retry automático.
6. **Não validado:** execução fora do Docker (host sem Python 3.12) e uso de GPU.
7. **Erros conceituais em `answer` não são detectados.** Na resposta sobre staking, o modelo escreveu "depositar ETH em nós de mineração"; o Ethereum não usa mineração desde a migração para proof-of-stake (2022). A trava cobre valores de mercado, não a correção do conteúdo.
8. **Fronteira `answered` × `requires_market_data` oscila:** "risco de manter SOL" recebeu resposta qualitativa sem plano, apesar da regra.

3.8 Aprendizados

- **Schema válido não é saída correta.** Formato e conteúdo precisam de verificações separadas; os testes unitários cobrem formato, mas qualidade semântica só aparece com o modelo real.
- **Em modelos pequenos, exemplo vale mais que regra.** O few-shot corrigiu o que regras explícitas não corrigiram, mas também introduziu vieses de cópia. O exemplo precisa ensinar o *processo* (extrair do texto), não só o *formato*.
- **Ajuste de prompt tem efeito colateral.** Cada correção alterou outros casos; sem um conjunto fixo de perguntas de regressão, as regressões teriam passado despercebidas.
- **O modelo não sabe que dia é hoje.** Contexto temporal precisa ser injetado; senão o modelo "corrige" o usuário com base no seu corte de treino.
- **Ambiguidade no prompt vira inconsistência na saída.** A instabilidade de `missing_information` vinha da redação da regra, não do modelo.
- **A ordem dos campos no schema é uma decisão de prompt.** O modelo gera o JSON em sequência; campos que condicionam outros (`answer_status` → `investigation_steps`) devem vir antes.
- **Nem toda falha de validação deve virar erro.** Omissões deriváveis (etapa de coleta) e conteúdo proibido localizado (uma frase com valor) podem ser corrigidos deterministicamente e registrados, preservando o restante da resposta.
- **Plano e resposta atendem públicos diferentes.** O plano serve às próximas etapas do sistema; o usuário precisa de uma resposta, mesmo que seja "não tenho esse dado nesta fase".

3.9 Pendências e insumos para a próxima fase

- **Conjunto de regressão de prompt:** transformar as 5 perguntas em suíte de avaliação automatizada, com critérios por campo, para medir cada mudança de prompt ou modelo.
- **Checagem determinística de ativos:** extrair tickers do texto com o catálogo de ativos das APIs (CoinGecko/CoinPaprika), sem depender do modelo.
- **Mapeamento `requested_metrics` → ferramentas:** o vocabulário `METRIC_VOCABULARY` é o ponto de partida do contrato com as APIs.
- **Retry/fallback** (`with_retry`, `with_fallbacks`) para `AnalysisOutputError`.
- **Avaliar outros modelos e GPU:** comparar qualidade e latência (ex.: modelos maiores ou com melhor aderência a JSON).
- **Reduzir o prompt:** a leitura de ~2.400 tokens domina a latência; avaliar `format_instructions` mais enxuto (o JSON schema completo) e o custo/benefício de cada exemplo few-shot, medindo com o benchmark.
- **Definir o modelo padrão** — feito: `gemma4:e2b` sem thinking (5.6). Revisar quando as etapas passarem a ser executadas, já que o E2B gera planos mais rasos.
- **Verificação de fatos conceituais:** com RAG ou fontes de referência, checar respostas `answered` (ex.: o erro "nós de mineração").
- **Substituir a trava heurística:** com dados reais disponíveis, valores em `answer` podem vir das ferramentas, com fonte e data, em vez de serem bloqueados.
- **Pergunta aberta:** `confidence` deve continuar sendo autoavaliação do modelo ou ser calculada a partir de sinais objetivos (ativos identificados, período presente, número de premissas)?

4 Fase 2 — Agente com ferramentas, roteamento, memória e base vetorial

Período	2026-10-19 a 2026-10-26
Status	Concluída
Ambiente	Linux (Manjaro), Docker Compose 5.5.1, Ollama 0.40.6, inferência em CPU (12 threads)
Modelos	`gemma4:e2b` (chat, thinking desligado, `num_ctx` 8192) e `nomic-embed-text` (embeddings)
Versões	`langchain-core` 1.6.12, `langchain-ollama` 1.1.0, `langchain-classic` 1.0.8, `langchain-chroma` 1.1.0, `chromadb` 1.5.9 (antes: `langchain-community` 0.4.2 + `faiss-cpu` 1.15.1), `fastapi` 0.150.0, `uvicorn` 0.54.0

4.1 Objetivo

Evoluir a chain da Fase 1 para um agente que:

- escolhe ferramentas e consulta dados reais (CoinPaprika);
- roteia tarefas e mantém contexto de conversa;
- busca conceitos numa base local (FAISS);
- produz respostas validadas.

Restrições:

- usar as APIs legadas exigidas pelo enunciado (`initialize_agent`, `ZERO_SHOT_REACT_DESCRIPT` `RouterChain`) sem comprometer a Fase 1;
- **nunca inventar dado de mercado**.

Os 10 critérios de aceite foram atendidos (`docs/stage-2-evidence.md`, seção 1).

4.2 Escopo

Dentro:

- cliente e ferramentas CoinPaprika;
- agente ReAct clássico;
- prompt PTCF com few-shot;
- chains sequencial, paralela e de conhecimento;
- roteamento moderno e legado;
- memória limitada;
- FAISS local;
- callbacks e logging;
- aplicação HTTP;
- testes e documentação.

Fora (enunciado): DeFiLlama e CoinGecko (a rota DeFi responde `unavailable`), memória persistente, planejamento autônomo.

4.3 Solução implementada

```
POST /agent  → CryptoAgent → initialize_agent(ZERO_SHOT_REACT) → ferramentas (CoinPaprika, FAISS) ┐
POST /route  → classificador (LLM | RouterChain legado | palavras-chave) → RunnableBranch          ├→
↪  grounding → CryptoReport
               ├ market_analysis: identificar → buscar → validar → analisar (sequencial)            │
               ├ asset_comparison: identificar → [ativos || global] → validar → comparar (paralela)  │
               ├ knowledge_retrieval: FAISS → resposta com base nos trechos                          │
               └ defi_analysis: indisponível                                                         ┘
POST /analyze, /analyze/batch, /analyze/stream → chain da Fase 1 (inalterada)
```

Detalhes por módulo em `docs/architecture.md`.

4.4 Decisões técnicas

Decisão	Alternativas	Motivo
APIs legadas via `langchain-classic`, isoladas em `app/legacy/`	Rebaixar para LangChain 0.x; usar só `create_agent`	No LangChain 1.x o legado saiu do pacote `langchain`. O `langchain-classic` 1.0.8 é compatível com o `langchain-core` 1.6.12 da Fase 1, sem rebaixar nada. Isolamento permite trocar depois
`RouterChain` só para **classificar**	`MultiPromptChain` completo	`MultiPromptChain` exige destinos `LLMChain`; nossos destinos são chains LCEL com API
Saída montada por **código** (grounding), não pelo LLM	Pedir ao ReAct um "Final Answer" em JSON	Modelo 2B + protocolo textual = JSON frágil. Fontes, horários e achados vindos das observações reais eliminam a chance de inventar esses campos
Verificação de valores da resposta contra as observações	Confiar no prompt	Na Fase 1 vimos que prompt não garante comportamento
Ferramentas de entrada única com normalização	Ferramentas estruturadas	O zero-shot só aceita uma string
`nomic-embed-text` via Ollama	sentence-transformers (torch)	Reaproveita o container, 274 MB, sem PyTorch (o disco esteve no limite na Fase 1)
Memória própria, limitada por turnos e caracteres, com marcação de dados históricos	`ConversationBufferWindowMemory`	O zero-shot não tem slot de memória e a janela não limita tamanho
`OLLAMA_NUM_CTX=8192`	Padrão 4096	O prompt do agente (cerca de 1.700 tokens antes das observações) mais histórico e observações passaria de 4096, com truncamento silencioso

Decisão	Alternativas	Motivo
Aplicação HTTP (FastAPI)	Scripts avulsos; CLI única	Decisão do usuário (seção 5.5)
Retry só para falhas transitórias	Retry geral	Resposta inválida com temperatura 0 tende a se repetir

4.5 Problemas encontrados e como foram resolvidos

4.5.1 Compatibilidade das APIs legadas

- **Investigação:** instalação limpa em `python:3.12-slim` com as versões da Fase 1 fixadas.
- **Resultado:** `initialize_agent`, `AgentType`, `LLMRouterChain`, `RouterOutputParser` e `ConversationBufferWindowMemory` importam de `langchain_classic`; FAISS de `langchain_community`.
- **Achado posterior:** o `langchain-community` emite aviso de **descontinuação** ("is being sunset"), e não existe `langchain-faiss` no PyPI. A dependência ficou restrita a `app/knowledge/store.py` e foi registrada como risco.

4.5.2 Comportamento real da CoinPaprika

Investigado com chamadas reais antes de codificar:

- `/search` sem barra final responde 301;
- id inexistente responde 404;
- **OHLCV histórico responde 402 no plano gratuito**, com `retry-after` de cerca de 12 dias;
- a cota é de 20 mil chamadas por mês;
- `/global` informa `last_updated` em epoch.

Consequências no design:

- 402 vira `plan_restricted`, sem retry;
- 429 só é repetido quando o `Retry-After` é de até 10 s;
- não há série histórica nesta fase (documentado no PTCF como limitação da fonte).

4.5.3 Primeira execução real do agente

O `gemma4:e2b` seguiu o protocolo ReAct logo na primeira execução:

- chamou `get_market_data("BTC")`, e a ferramenta resolveu o símbolo para `btc-bitcoin`;
- respondeu com o valor real, a fonte e o horário;
- a verificação de grounding confirmou os números.

Tempo total: 48 s (30 s só para o modelo ler o prompt de cerca de 1.700 tokens).

4.5.4 Comparações erradas do modelo: três iterações

#	Mudança	Resultado real
1	Resumo livre do LLM (valores verificados)	Valores certos, conclusão errada: “o Bitcoin apresentou a maior variação negativa” (o ETH caiu mais); em outra execução, “o Ethereum apresenta capitalização maior”
2	Rankings (variação 24h e market cap) calculados no código e enviados ao modelo	**Repetiu o erro**: “o Bitcoin apresentou a maior queda” com o ranking certo no prompt
3	Na rota de comparação, a conclusão (`answer`) é montada pelo código; o resumo do LLM é descartado; os `key_points` verificados ficam	Correto: “SOL -0,79% > BTC -0,84% > ETH -1,07%” (2026-10-26)

Aprendizado: a verificação de grounding confere *valores*, não *conclusões*. Para modelos pequenos, ordenação e comparação devem ser feitas pelo código.

Correções menores no grounding, achadas com respostas reais:

- “$1.638 trilhões” (correto) era removido, porque “1.638” era lido como milhar;
- “caiu 1,11%” não batia com -1,11, porque o sinal vem da palavra;
- o regex capturava a vírgula final da frase;
- o LLM colocava preços no campo `uncertainties`, que não era verificado.

4.5.5 Formato da aplicação: scripts → CLI única (descartada) → HTTP

- **Inicial:** um script por tarefa em `scripts/`, seguindo o “executar por scripts Python” da Fase 1.
- **Questionamento do usuário:** os vários `__init__.py` seriam consequência dos scripts? Resposta: os `__init__.py` são padrão de pacote Python e ficam em qualquer formato. Só o `scripts/__init__.py` vinha da escolha de scripts. O desvio real eram 8 pontos de entrada com inicialização repetida, sem entrada única e sem instalação.
- **Proposta:** CLI única com subcomandos. Aprovada e depois **cancelada pelo usuário** antes da implementação.
- **Decisão final (usuário): serviço HTTP com FastAPI.**
  - `app/main.py` com contêiner de dependências criado no ciclo de vida da aplicação;
  - 10 endpoints;
  - `scripts/` removido;
  - o benchmark virou módulo de manutenção (`python -m app.services.benchmark_runner`), por levar de 10 a 40 min.
- **Ajuste no stream:** a primeira versão de `/analyze/stream` acumulava os parciais e só enviava no fim. Foi corrigida para enviar cada parcial assim que gerado.

4.5.6 Exemplos do Swagger

O Swagger mostrava `"question": "string"`. Foram criados exemplos reais e nomeados por cenário (`app/api/examples.py`). Um teste valida cada exemplo contra o modelo da requisição, para que não fiquem inválidos se os modelos mudarem.

4.5.7 Retry de LLM

As etapas JSON das chains ganharam `with_retry` para falhas de conexão com o Ollama (duas tentativas). Erros de validação não são repetidos. Ollama inacessível após o retry responde `503`. O agente ReAct não pode usar o wrapper, porque o legado exige `BaseLanguageModel`; nele, a falha vira `CryptoReport failed`.

4.5.8 Troca de FAISS por Chroma

- **Decisão do usuário (2026-10-26):** usar Chroma como base vetorial no lugar do FAISS. O enunciado pedia FAISS; a mudança é um desvio deliberado e registrado.
- **Registro completo da decisão:** docs/vector-store-decision.md (contexto, problemas do FAISS, motivação, alternativas, consequências, validação e como reverter).
- **Por que mudar (problemas observados com o FAISS durante a fase):**
  1. `langchain-community` em descontinuação, com aviso em toda execução; não existe `langchain-faiss`;
  2. persistência via pickle, que exigia `allow_dangerous_deserialization=True`;
  3. sem atualização incremental nem filtro por metadados, o que pesaria com o crescimento previsto da base (memória de longo prazo).
- **Alternativas avaliadas:**
  - manter FAISS via `langchain-community`;
  - FAISS puro;
  - Chroma como servidor;
  - Qdrant, Weaviate, Milvus, pgvector.

  Escolhido: **Chroma embutido persistente** (pacote dedicado, sem pickle, CRUD e metadados nativos, sem serviço extra).
- **Investigação:** `langchain-chroma` 1.1.0 (pacote dedicado) + `chromadb` 1.5.9 instalam com `langchain-core` 1.6.12 fixado.
- **Custo medido (2026-10-28, `python:3.12-slim` com os `requirements.txt` reais):** FAISS 305 MB / 63 pacotes → Chroma 564 MB / 108 pacotes (+259 MB). O `chromadb` traz `onnxruntime`, `grpcio`, `kubernetes` e OpenTelemetry, que o projeto não usa diretamente.
- **Modo de uso:** Chroma embutido persistente (`PersistentClient`) em `data/chroma`, montado no Compose, sem serviço extra; coleção com distância de cosseno; telemetria anônima desligada.
- **Reindexação:** a coleção é apagada e recriada, com ids determinísticos por trecho, para não acumular duplicatas.
- **Efeitos colaterais positivos:**
  - removeu o `langchain-community`, cuja descontinuação estava registrada como risco em 5.1;
  - removeu o carregamento do índice via pickle (`allow_dangerous_deserialization`), um risco de segurança;
  - as interfaces (`KnowledgeSearcher`, ferramenta, rota, endpoints) não mudaram, porque o acesso à base já estava isolado em `app/knowledge/store.py`.
- **Validação real:** ingestão em 3,5 s; as 4 buscas de teste trouxeram o documento correto em 1º lugar; rota e agente responderam a partir dos trechos; 199 testes

offline e o teste de integração passaram (`docs/stage-2-evidence.md`, seção 9).
- **Achado:** achados de conhecimento duplicados (dois trechos do mesmo arquivo), corrigidos com deduplicação.

4.6 Evidências e métricas

Detalhes em `docs/stage-2-evidence.md`.

Verificação	Resultado
Testes offline	196 passed (~4 s)
Integração + rede	11 passed (340 s)
`GET /health/coinpaprika`	7/7
Agente ReAct (ETH)	`complete`, 32 s, valores e horários reais
Continuação na mesma sessão	Nova consulta à API, sem reaproveitar valor antigo
Comparação de 3 ativos (paralela)	`complete`, 67 s, conclusão correta
`RouterChain` legado	Classificou corretamente
Rota DeFi	`unavailable` explícito

4.7 Limitações conhecidas

Ver `docs/reliability-decisions.md` (seção 6) e `docs/stage-2-evidence.md` (seção 7). As principais:

- a verificação confere valores, não conclusões, fora da rota de comparação;
- percentuais são comparados em módulo;
- a latência em CPU é de 15 a 70 s;
- a memória vive só no processo;
- na pergunta “Esse ativo está caro?”, o agente não pediu o ativo como no exemplo few-shot.

4.8 Aprendizados

- **Investigar a API real antes de codificar** evitou suposições erradas: o 301 do `/search`, o 402 do histórico e o `retry-after` de dias mudaram o design de retries.
- **Montar a saída por código** (fontes, horários, achados, status) é mais confiável que pedir JSON a um modelo pequeno em protocolo textual.
- **Ordenação e comparação não devem ser delegadas a modelos pequenos.** Nem com o ranking no prompt o modelo acertou.
- **Janela de contexto é requisito funcional:** o padrão de 4.096 tokens truncaria o agente em silêncio.
- **Isolar o legado** reduziu o custo de cumprir o enunciado. A troca para LangGraph ou `create_agent` afeta só `app/legacy/`.
- **Exemplos de documentação também são código:** sem teste, eles ficam inválidos.

4.9 Pendências e insumos para a próxima fase

- Integrar DeFiLlama (rota `defi_analysis`) e CoinGecko (séries históricas, que o plano gratuito da CoinPaprika não oferece).
- Migrar o agente para LangGraph ou `create_agent` com tool calling nativo, com ferramentas de argumentos tipados e retry no LLM.

- Executar o plano da Fase 1 (`requested_metrics` → ferramentas), o planejamento hierárquico.
- Memória persistente (sessões) e de longo prazo.
- Substituir o `langchain-community` — resolvido com a troca para Chroma (5.8).
- Avaliar se o few-shot de “informação ausente” precisa de reforço, ou de uma checagem determinística de “sem ativo”.

5 Fase 3 — Memória conversacional e raciocínio contextual

Período	2026-10-28 a 2026-10-29
Status	Concluída
Ambiente	Linux (Manjaro), Docker Compose, Ollama, CPU (12 threads)
Modelos	`gemma4:e2b` (thinking desligado) e `nomic-embed-text`
Versões novas	`mem0ai` 2.2.1 (mantidos: `langchain-core` 1.6.12, `chromadb` 1.5.9, `fastapi` 0.150.0)

5.1 Objetivo

Transformar a aplicação num assistente conversacional que:

- mantém o contexto entre mensagens e entre sessões, com persistência em SQLite e memória semântica no Chroma;
- interpreta continuações, correções e referências a resultados anteriores;
- não apresenta dado antigo como atual;
- preserva as Fases 1 e 2.

5.2 Escopo e adaptações do enunciado

O enunciado foi **adaptado por decisão do usuário** antes da implementação:

1. **Nada depreciado.** O `ConversationBufferMemory` (exigido na seção 5) está depreciado desde o LangChain 0.3.1, com remoção na 2.0, e **não foi usado**. Histórico e janela são próprios, sobre SQLite.
2. **Framework de memória especializado.** A seção 12 pedia só comparar. O usuário pediu para usar um: LangMem primeiro; depois da avaliação, Mem0.

Mantido do enunciado:

- API existente;
- LangChain na orquestração;
- SQLite como fonte da verdade e Chroma para memória semântica;
- 7 intenções;
- 8 cenários;
- documentação pedida.

Fora do escopo: CoinGecko e DeFiLlama (DeFi continua `unavailable`) e autenticação.

5.3 Solução implementada

```
POST /conversations/{id}/messages
 → trava por sessão → SQLite (mensagem) → intenção (regras + LLM) → resolver (contexto ativo, análises)
 → contexto (janela + resumo + memórias Mem0 da sessão, com orçamento) → chains LCEL da Fase 2 ou
↪  comparação com resultado anterior
 → SQLite (resposta, análise, contexto ativo) + episódio no Mem0 → segundo plano: resumo, reindexação,
↪  extração semântica
```

Módulos novos:

- `app/persistence/` (SQLite);
- `app/memory/{short_term,long_term}.py`;

- `app/intent/` (schemas, classificador, resolver);
- `app/conversation/` (`context_builder`, serviço);
- `app/api/conversation_routes.py`.

Detalhes em `docs/architecture.md` (seção 8) e `docs/memory-strategy.md`.

5.4 Decisões técnicas

Decisão	Alternativas	Motivo
Não usar `ConversationBufferMemory`	Usar isolado em `app/legacy/` (como o ReAct da Fase 2)	Diretriz do usuário: depreciado (remoção na 2.0); `InMemoryChatMessageHistory` também depreciado
Mem0 para a memória longa	LangMem, Letta, código próprio	Spikes (seção 5.1): único sem depreciação, Chroma e SQLite nativos, isolamento por `run_id` testado
Resumo e janela em código próprio (LCEL)	`summarize_messages` do LangMem	LangMem descartado (depreciação na dependência); o Mem0 não resume conversa
Fatos que sobrevivem ao resumo extraídos dos **metadados**	Deixar tudo para o LLM	Lição das Fases 1 e 2: modelo pequeno erra e omite. Resultados anteriores, correções e entidades não podem depender dele
Regras determinísticas prevalecem sobre o LLM na intenção	Só LLM	Período, correção, ativos e referência são detectáveis por regra com mais confiabilidade que o e2b
Episódios com `infer=False` e extração semântica em segundo plano	Extração em todo turno	Extração do Mem0 leva de 68 a 89 s; episódios precisam ser exatos
Filtro anti-valores de mercado na memória	Confiar no prompt	No spike, o Mem0 guardou “ETH a US$ 2.329,72” como fato
`MemoryBackend` (interface) + `LazyMemoryBackend`	Mem0 direto	Testes offline sem rede; API sobe sem Ollama; troca de backend localizada
Conclusões numéricas (diferença atual × anterior) feitas pelo código	Pedir ao LLM	Lição da Fase 2: o modelo inverte comparações

5.5 Problemas encontrados e como foram resolvidos

5.5.1 Escolha do framework de memória: três spikes

Candidato	Achado	Decisão
LangMem 0.0.30	Dependências compatíveis; `summarize_messages` funcionou (22 s); **`trustcall` importa `langgraph.constants.Send`, depreciado no LangGraph 1.0** (o aviso aparece em qualquer uso); 1 de 3 extrações inventou memória	Pela política combinada, parar e consultar → usuário pediu Mem0/Letta
Letta	Documentação oficial: a imagem Docker "não é mais mantida nem suportada"; Postgres + pgvector; laço de agente próprio	Descartado sem execução
Mem0 2.2.1	Sem depreciação; Chroma e SQLite; isolamento por `run_id` (ok); extração **lenta** (68–89 s) e **guardou preço como fato**	**Escolhido**, com mitigações (seção 4)

Registro completo: `docs/memory-decision.md`.

5.5.2 30 dias da CoinPaprika

O plano gratuito devolve `percent_change_30d = 0` (e `1y = 0`) para todos os ativos, verificado com BTC, ETH e SOL. Na Fase 2 esse 0 chegava ao modelo como dado. Agora:

- 0 exato vira "indisponível";
- pedidos de 30 dias recebem limitação explícita (`partial`), com as variações reais de 24h e 7d.

5.5.3 Ajustes de regras e testes

- "Qual o preço atual?" e "Considere os últimos 30 dias" **sem contexto** caíam em "pesquisa geral". Corrigido: viram pedido de esclarecimento.
- **Referência ambígua** a resultado anterior: o contexto ativo resolvia sozinho. Agora só resolve com referência explícita ("os dois", "them"); sem ela, pergunta.
- **Teste de depreciação** com `importlib.reload` recriava classes de exceção e quebrava outros testes. Movido para um subprocesso limpo.

5.5.4 Problemas revelados pela execução real

Sintoma	Correção
Declaração de preferência rodou de novo a análise DeFi	Preferência só é confirmada e enviada à extração

Sintoma	Correção
Resumos acumulavam na memória longa (v1, v2, v3)	`replace_episode`: um resumo por sessão no Chroma

5.6 Evidências e métricas

Detalhes em `docs/conversation-evaluation.md`.

Verificação	Resultado
Testes offline (Fases 1–3)	**260 passed**
Integração Fase 3 (Ollama + Mem0 + CoinPaprika reais)	1 passed, 537 s (A, D, B, preferência, reinício, C, isolamento)
Integração completa (Fases 1–3)	**12 passed** em 702 s (5 Fase 1 + 6 Fase 2 + 1 Fase 3), sem regressão
API real com `docker compose restart app`	Sessão retomada (8 mensagens em ordem, contexto ativo restaurado); continuação, resultado anterior e isolamento corretos; sessão inexistente → 404
Memória semântica real	"O usuário prefere que as análises… focadas em risco" recuperada com score 0,617; nenhum valor de mercado gravado
Tempo por turno (CPU)	7 s (resultado anterior) a cerca de 70 s; 8 s para confirmar uma preferência

5.7 Limitações conhecidas

- **Disputa pelo Ollama:** a extração em segundo plano compete com as requisições ("Analise SOL" levou 177 s).
- **"Resultado anterior"** pega a análise mais recente com os mesmos ativos, o que pode ser a do turno anterior (diferença de 0%).
- **Memórias semânticas** às vezes gravadas em inglês pelo Mem0.
- **Sem autenticação**; `session_id` não é credencial.
- **Travas por sessão** valem por processo, não entre vários workers.
- **Contagem de tokens aproximada**, porque o Ollama não expõe o tokenizer.

5.8 Aprendizados

- **Depreciação também vem de dependências transitivas:** o LangMem em si não usa nada depreciado, mas o `trustcall` sim. Medir no spike, com a pilha real, evitou adotar algo com prazo de remoção.
- **Frameworks de memória pressupõem modelos fortes.** Com um modelo de 2B em CPU, extração por LLM é lenta e precisa de filtros determinísticos. O que é crítico (episódios, resultados anteriores) não deve depender dela.
- **A memória pode reintroduzir dado desatualizado.** Um preço guardado como "fato" seria reapresentado depois; a política de retenção precisa proibir isso explicitamente.
- **Regras primeiro, LLM depois,** na interpretação de continuações: período, correção e ativos citados são detectados com mais confiabilidade por regras do que pelo modelo pequeno.

- **Testar com a API real e reinício real** revelou dois comportamentos (preferência e resumos acumulados) que os testes com dublês não pegaram.

5.9 Pendências e insumos para a próxima fase

- **Fila ou prioridade para o Ollama**, para a manutenção em segundo plano não atrasar as respostas (ou extração só em horários ociosos).
- **Heurística de "resultado anterior"**, distinguindo "o de agora há pouco" de "o de antes".
- **Autenticação e `user_id` real** no Mem0, se houver mais de um usuário.
- **Migração do agente da Fase 2** para LangGraph (`create_agent` com checkpointing), que unificaria `/agent` e `/conversations`.
- **DeFiLlama e CoinGecko:** as intenções `defi_analysis` e `historical_analysis` já preservam o contexto para quando houver fonte.

6 Fase 4 — Ferramentas externas e integração com o RAG existente

Período	2026-11-04 (horário local; as execuções reais estão em UTC, 2026-11-05 00:30–01:30)
Status	Concluída
Ambiente	Linux (Manjaro), Docker Compose, Ollama, CPU (12 threads)
Modelos	`gemma4:e2b` (thinking desligado) e `nomic-embed-text`
Versões novas	`sqlglot` 30.27.0. Mantidos: `langchain-core` 1.6.12, `langchain-classic` 1.0.8, `chromadb` 1.5.9, `mem0ai` 2.2.1, `fastapi` 0.150.0

6.1 Objetivo

Ampliar o agente ReAct existente para combinar:

- dados de mercado ao vivo (CoinPaprika, já existente);
- consultas estruturadas sobre um banco SQLite de snapshots, via **Text-to-Query** com SQL validado;
- recuperação de documentos na coleção Chroma existente, via **Retrieval Agent** com **Query Rewriting**;
- cálculo determinístico.

Tudo sem reconstruir a ingestão do corpus, sem novos provedores e preservando as Fases 1–3.

Critério de conclusão:

- ferramentas catalogadas e classificadas;
- banco populado com dados reais;
- SQL seguro e validado;
- ao menos uma falha real de recuperação diagnosticada e comparada antes/depois da reescrita;
- testes executados;
- documentação atualizada.

6.2 Escopo

Dentro:

- catálogo de ferramentas;
- banco SQLite de snapshots (schema, coleta, deduplicação, cobertura);
- Text-to-Query (geração, validação por parser, execução protegida, status explícitos);
- Retrieval Agent e Query Rewriting;
- integração ao ReAct;
- endpoints de operação e avaliação;
- testes unitários e de integração;
- documentação.

Fora (e por quê):

- **CoinGecko e DeFiLlama:** proibidos pelo enunciado nesta fase;
- **Multi-Query e HyDE:** só Query Rewriting, por enunciado;
- **mudança no chunking ou reindexação:** proibido ("não reconstruir a ingestão"). Os problemas de chunking encontrados ficaram registrados como limitação;
- **agendador de coleta:** o enunciado pede gatilho simples;
- **migração do ReAct legado:** o enunciado pede integrar ao agente existente.

6.3 Solução implementada

```
POST /agent → ReAct (PTCF, regra 6: pergunta → ferramenta)
   ├─ CoinPaprika ao vivo: search_crypto_asset, get_market_data, get_market_details, get_global_market
   ├─ query_market_database → TextToQueryService → SQL validado → data/market.sqlite3
   ├─ search_knowledge_base → RetrievalAgent (original → [fraca] reescrita → evidência) →
↪  crypto_knowledge
   └─ calculate_percent_change (local)
POST /market-data/ingest → MarketDataIngestor → CoinPaprikaClient (reuso) → SQLite
```

Módulos novos:

- `app/market_data/` (`store`, `ingestion`, `sql_guard`, `text_to_query`);
- `app/retrieval/` (`rewriter`, `agent`);
- `app/tools/{market_db_tool, retrieval_tool, calculation_tools}.py`;
- `app/api/phase4_routes.py`.

Detalhes:

- `docs/tools-catalog.md`;
- `docs/text-to-query.md`;
- `docs/retrieval-agent.md`;
- `docs/architecture.md` (seção 9).

6.4 Decisões técnicas

Decisão	Alternativas	Motivo
Banco SQLite **separado** (`data/market.sqlite3`)	Tabelas no SQLite das conversas	Enunciado: não misturar responsabilidades; apagar ou recriar um não afeta o outro
Schema = campos reais de `MarketTicker`, `NULL` quando ausente	Schema "ideal" com OHLCV histórico	Não inventar métricas; histórico OHLCV dá 402 no plano gratuito
Deduplicação por `(asset_id, source_updated_at)` (índice único parcial)	Por horário local	O mesmo dado da fonte coletado duas vezes não é observação nova
Validação em **duas camadas**: `sqlglot` (antes) + `mode=ro` + `set_authorizer` + `progress_handler` (durante)	Só regex; só parser	O enunciado pede parser. O authorizer pega o que o parser deixa passar (`random()`, comprovado em teste)

Decisão	Alternativas	Motivo
Literais do SQL do LLM viram parâmetros ?	Executar o texto do LLM	Enunciado: preferir parametrização; o texto executado não carrega valores
Período detectado por **regra** (`find_period`, Fase 3), não pelo LLM	Campo do LLM	Lição das Fases 1–3: regra é mais confiável que o modelo pequeno
`insufficient_coverage` **sem executar** SQL	Executar e devolver o que houver	Responder “90 dias” com 5 minutos de dados seria enganoso
Ferramenta de retrieval com o **mesmo nome** da Fase 2 (`search_knowledge_base`)	Nome novo	Prompt, regras e grounding já a conhecem; formato compatível
Reescrita só quando a busca original é fraca (`auto`, distância > 0,33)	Sempre reescrever	Enunciado: evitar reescrita sem valor; economiza 10–13 s de LLM por busca boa
Evidência = distância à consulta usada ≤ 0,38 **+ palavra âncora** nos trechos da reescrita	Menor distância vence; distância à pergunta original	As duas alternativas foram testadas com dados reais e falharam (seção 5.2)
View `snapshot_history` com `symbol/name`	Exigir JOIN do modelo	O `gemma4:e2b` errou o JOIN (seção 5.3)

6.5 Problemas encontrados e como foram resolvidos

6.5.1 Primeira escolha de exemplo few-shot contaminava o experimento

- **Sintoma:** o exemplo do prompt de reescrita era justamente “recompensa cai pela metade → halving”, um dos casos avaliados.
- **Correção:** o exemplo foi trocado por um tema fora do corpus (“rug pull”), antes das medições finais.
- **Efeito:** sem o exemplo, o modelo não escreve mais “halving”, mas a reescrita ainda melhora o ranking (`docs/phase-4-evidence.md`, seção 1.2). O resultado registrado é o honesto, sem a ajuda do exemplo.

6.5.2 Política de escolha entre busca original e reescrita (três versões)

Versão	Regra	Falha observada com dados reais
1	Fica a busca com o melhor trecho mais próximo	“Queima de moedas” (fora do corpus): a reescrita acrescentou termos genéricos e levou a distância de 0,437 para 0,183 em trechos **irrelevantes**. Isso fabricaria evidência

Versão	Regra	Falha observada com dados reais
2	Relevância = distância à **pergunta original** (piso de 0,42; ranking da reescrita)	Funcionou no halving, mas em “impermanent loss” os trechos de DeFi trazidos pela reescrita ficaram **mais longe** da pergunta (0,425–0,455) que trechos errados (0,387) e foram descartados. Usar essa distância para **comparar** as buscas falha por construção: a original é ranqueada por ela
3 (final)	Distância à consulta usada ≤ 0,38 + **palavra âncora** da reescrita presente no trecho (prefixo, sem termos genéricos)	Barra a evidência fabricada (queima) e aceita a tradução de vocabulário (halving). Limitações na seção 7

6.5.3 Erros semânticos do gerador SQL

Sintoma (execução real)	Causa	Correção
`... WHERE symbol IN ('ADA', ..., 'USDT')` sem filtro pedido, **omitindo XRP** no ranking por market cap	A lista de símbolos estava no prompt, e o modelo a copiou como filtro	Prompt passa só a quantidade de ativos e a janela; regra “só filtre por symbol quando a pergunta citar o ativo”
`SELECT AVG(price_usd) FROM market_snapshots WHERE symbol = 'BTC'`	`market_snapshots` não tem `symbol` (exige JOIN). **O validador recusou** antes da execução (`invalid_query`)	View `snapshot_history` com `symbol`/`name`, mais um exemplo de média histórica

Depois das correções, a bateria final de 9 perguntas teve 9 status corretos (`docs/phase-4-evidence.md`, seção 2.1).

6.5.4 CTE recursiva negada pelo authorizer com erro genérico

- **Sintoma:** o teste de timeout dava `execution_error` em vez de `timeout`.
- **Causa:** o SQLite pede autorização da ação `SQLITE_RECURSIVE`, e “not authorized” chega como `DatabaseError`, não como `OperationalError`.
- **Correção:** a ação recursiva (só leitura) passou a ser permitida, e “not authorized” é tratado em qualquer `DatabaseError`.

6.5.5 Agente real: pergunta combinada sem documento e entrada resumida

Achados da 1ª rodada com o agente real (`docs/phase-4-evidence.md`, seção 4.2):

- **Pergunta combinada (mercado + PoW × PoS):**

  - **Sintoma:** o agente só chamou `get_market_data`, explicou o conceito com o conhecimento do modelo e devolveu `complete`.
  - **Correção:** regra no PTCF e guarda determinística: pedido conceitual sem trecho recuperado vira `partial` com incerteza.
  - **Resultado:** o modelo continua sem buscar, mas a falta de base documental agora fica explícita.
- **Halving pelo agente:**
  - **Sintoma:** o ReAct enviou "recompensa dos mineradores", sem "cai pela metade". A busca não acionou a reescrita, e a resposta disse que o texto não tratava da queda.
  - **Correção:** a descrição da ferramenta pede a pergunta completa.
  - **Resultado:** reescrita aplicada e resposta correta citando o halving.
- **"Maior que 5%" removido da resposta:**
  - **Causa:** o limiar do filtro não estava nas observações.
  - **Correção:** os `params` do SQL entram na observação.

6.5.6 Regressão completa: duas falhas reais

1ª execução da suíte de integração completa: **16 passed, 2 failed** (`docs/evidence/phase-4/regression-run1.log`).

- **Fase 3 (`test_live_multi_turn_restart_and_isolation`):**
  - **Causa:** mudança da fonte, não do código. Em 2026-11-05 a CoinPaprika passou a devolver `percent_change_30d` real (BTC -6,00%), e a resposta trouxe "Variação 30d" em vez da limitação "30 dias", que o teste exigia.
  - **Correção:** o teste aceita os dois comportamentos corretos; no Text-to-Query, 30 dias passou a ser janela do provedor (`PROVIDER_WINDOWS_DAYS = {1, 7, 30}`); a documentação que dizia "30d sempre NULL" foi corrigida.
- **Fase 4 (halving):**
  - **Causa:** com o mesmo prompt e temperatura 0, o Ollama gerou outra reescrita, acrescentando "bitcoin ethereum". O trecho de Ethereum subiu para 1º.
  - **Correção:** `strip_unrequested_assets` remove da reescrita os ativos que a pergunta não cita, e o teste passa a exigir melhora para o top-2.
  - **Medição:** em 5 execuções seguidas, o trecho ficou em 1º quatro vezes e em 2º uma vez.

6.5.7 Testes rodavam o código antigo da imagem

- **Sintoma:** `ModuleNotFoundError` para os módulos novos.
- **Causa:** a imagem copia o código no build, e o Compose só monta `./data`.
- **Correção:** durante o desenvolvimento, os testes rodaram com `-v $PWD:/app`. A imagem final foi reconstruída.

6.6 Evidências e métricas

Detalhes e saídas completas: `docs/phase-4-evidence.md` (dados brutos em `docs/evidence/phase-4/`).

Verificação	Resultado
Testes offline (Fases 1–4)	**312 passed** (52 novos da Fase 4)
Integração Fase 4 (Ollama + CoinPaprika + Chroma reais)	**6 passed** em 169,5 s

Verificação	Resultado
Integração completa (Fases 1–4), 1ª execução	16 passed, 2 failed (seção 5.6)
Integração Fases 3 e 4 após correções	**7 passed** em 595,5 s
Coleta real	10 ativos inseridos; repetição imediata = `duplicates`; 3ª coleta: 8 novos + 2 duplicados (19 snapshots)
Text-to-Query real (9 perguntas)	9 status corretos: 5 `ok`, 1 `no_matching_records`, 1 `insufficient_coverage`, 2 `unsupported`
Diagnóstico de recuperação (halving)	Trecho relevante: 3º → 1º lugar (MRR 0,33 → 1,0); em 5 execuções seguidas: 1º em 4, 2º em 1
Agente real (5 perguntas)	Ferramenta certa nas 4 perguntas de um só tipo (mercado, banco, conceito, halving com reescrita); a combinada saiu `partial` (sem busca documental, declarada)

6.7 Limitações conhecidas

- **Chunking existente:**
  - `risks.md#12` só tem o título e aparece em muitas buscas;
  - `defi-metrics.md#6` dilui a frase sobre perda impermanente.

  Reescrever a consulta não corrige isso.
- **Distância não é relevância:** trechos genéricos ficam perto de perguntas fracas, e os limiares são empíricos (corpus de 18 trechos).
- **A palavra âncora é heurística:** palavras de domínio ("liquidez", "perda") deixam passar trechos só parcialmente relevantes (caso impermanent loss).
- **Histórico de mercado depende de coletas reais:** perguntas de 90 dias ou mais dão `insufficient_coverage`.
- **Saída do LLM não é estritamente determinística** em CPU, mesmo com temperatura 0: a reescrita variou entre execuções. Os testes de integração usam critérios de melhora, não igualdade.
- **O SQL do `gemma4:e2b` é correto em filtros e rankings simples;** consultas temporais complexas são frágeis. A validação garante segurança, não semântica.

6.8 Aprendizados

- **Medir antes de calibrar:** os limiares e a política final só saíram das distâncias reais. As duas políticas "óbvias" (menor distância; distância à pergunta original) falharam em casos concretos.
- **Reescrita também introduz risco:** termos genéricos acrescentados pelo LLM criam relevância aparente. A transformação de consulta precisa de uma checagem de que a evidência tem a ver com o conceito.
- **Few-shot pode contaminar a avaliação:** um exemplo igual ao caso de teste mascara o comportamento real do modelo.
- **Parser e authorizer se complementam:** cada um pegou casos que o outro não pegaria (função conhecida pelo parser, mas negada pelo SQLite; coluna inexistente recusada antes de chegar ao banco).
- **Contexto demais atrapalha o modelo pequeno:** a lista de ativos no prompt virou filtro indevido.

- **Fontes externas mudam:** a premissa “30d = 0 no plano gratuito”, verificada na Fase 3, deixou de valer uma semana depois. Testes de integração precisam aceitar os dois comportamentos legítimos, e a documentação deve datar esse tipo de fato.

6.9 Pendências e insumos para a próxima fase

- **Chunking por item de lista** em `risks.md` e uma seção própria para perda impermanente, se uma fase futura permitir mexer na ingestão.
- **Coleta agendada** para formar histórico (hoje é manual).
- **Cobertura por ativo** no Text-to-Query.
- **Migração do ReAct legado** para tool calling nativo (LangGraph), com ferramentas de entrada estruturada no lugar de texto único.
- **DeFiLlama/CoinGecko:** `unsupported` no Text-to-Query e `unavailable` nas rotas DeFi indicam onde entram.

7 Fase 5 — Agente integrado, verificação de respostas, avaliação e interface

Período	2026-11-05 a 2026-11-08
Status	Concluída
Ambiente	Linux (Manjaro), Docker Compose, Ollama, CPU (12 threads)
Modelos	`gemma4:e2b` (thinking desligado; agente, verificação e juiz) e `nomic-embed-text`
Versões novas	`langchain` 1.4.9 (com LangGraph 1.2.20), `langchain-core` 1.6.12 → **1.7.6**, `langsmith` 0.14.17 (já era dependência), `streamlit` 1.65.0

> Este diário foi consolidado no fim do projeto (2026-11-08) a partir de `docs/agent-integration.md`, `docs/response-verification.md`, `docs/langsmith-evaluation.md`, das evidências em `docs/evidence/phase-5/` e `docs/evidence/final/` e do histórico do git. As fases anteriores foram escritas durante o desenvolvimento.

7.1 Objetivo

Unir as capacidades das Fases 1 a 4 num **único agente** que:

- escolhe e combina ferramentas de API (CoinPaprika), SQL (snapshots), RAG (Chroma) e cálculo;
- usa o histórico persistente da Fase 3;
- **verifica cada resposta antes da entrega**: consistência factual contra a evidência real (NLI) e coerência lógica, com no máximo uma correção;
- é rastreado e avaliado (LangSmith, dataset versionado, avaliadores de resposta e de trajetória);
- tem uma interface Streamlit que só consome a API.

7.2 Escopo

Dentro: agente com tool calling nativo, ferramentas estruturadas, adaptador de histórico, serviço de verificação, política de correção, dataset e avaliadores, runner local e LangSmith, interface Streamlit, testes e documentação.

Adaptado por decisão do autor (2026-11-05):

Enunciado	Implementado	Motivo
`create_openai_tools_agent` + `AgentExecutor`	`create_agent` (pacote `langchain`)	APIs do pacote legado `langchain-classic`; `create_agent` é o substituto oficial do LangChain 1.x

Enunciado	Implementado	Motivo
`RunnableWithMessageHistory`	`SessionHistoryAdapter`	Depreciado no `langchain-core` 1.7.6 (“Use LangGraph’s built-in persistence instead”)
NLI com Transformers (`mDeBERTa-v3-base-mnli-xnli`)	NLI com `gemma4:e2b` via Ollama	Sem torch e transformers (cerca de 2 GB, com 5,3 GB livres) e um único motor de inferência

Fora: autenticação, CoinGecko e DeFiLlama.

7.3 Solução implementada

```
POST /conversations/{id}/agent
 └─ IntegratedAgentService (trace "integrated.turn")
     ├─ trava por sessão → grava a pergunta (SQLite)
     ├─ SessionHistoryAdapter: resumo + memórias Mem0 + janela (só leitura)
     ├─ create_agent: gemma4:e2b ⇄ 7 ferramentas estruturadas; trajetória registrada
     ├─ VerificationService: afirmações → evidência → checagem numérica | NLI → coerência → status
    ├─ [política] 1 correção com as próprias observações → 2ª verificação → resposta cautelosa se falhar
     └─ grava a resposta e a análise → manutenção da Fase 3 em segundo plano
```

Módulo	Responsabilidade
`app/integrated/`	Agente (`create_agent`), ferramentas estruturadas que delegam às implementações das Fases 2 e 4, adaptador de histórico e o turno
`app/verification/`	Extração de afirmações, evidência, NLI, coerência lógica, política de correção
`app/evaluation/`	Alvo, avaliadores (resposta e trajetória) e runner local/LangSmith
`evaluation/phase5_dataset.json`	Dataset versionado (19 exemplos, 9 categorias)
`ui/`	Interface Streamlit e cliente HTTP

Detalhes em `docs/agent-integration.md`, `docs/response-verification.md` e `docs/langsmith-evaluation.md`.

7.4 Decisões técnicas

Decisão	Alternativas	Motivo
`create_agent` sem checkpointer	Checkpointer do LangGraph	O SQLite da Fase 3 continua a única fonte da verdade da conversa; o adaptador só lê e o serviço grava uma vez por turno, sem duplicação
Manter o `gemma4:e2b`	Trocar de modelo para ter tool calling	O spike mostrou tool calling nativo funcionando pelo Ollama (bind, seleção, execução e incorporação)
Ferramentas estruturadas que **delegam**	Reescrever as ferramentas	Nenhuma regra de negócio duplicada; o contrato de observação da Fase 2 é mantido
Extração de afirmações determinística (por frase)	Extração por LLM	Mais uma chamada de 5 a 15 s por turno em CPU, para respostas que já são curtas
Números checados por código, não pela NLI	Tudo pela NLI	Lição das Fases 1 a 4: o modelo de 2B erra comparações numéricas
Seleção de até 3 evidências por afirmação, por pontuação	Comparar todas com todas	Custo de NLI proporcional ao número de afirmações, não ao produto
`neutral` ≠ `contradiction`; NLI indisponível → `verification_failed`	Tratar falha como contradição	Falta de evidência não é evidência contrária; falha de infraestrutura não pode condenar a resposta
No máximo 1 correção, sem laço	Corrigir até passar	Custo previsível (cada correção é mais uma execução do agente) e resposta cautelosa explícita quando não passa
Juiz LLM = `gemma4:e2b` local, junto com avaliadores determinísticos	Juiz externo	Sem dependência de modelo pago; o viés de autoavaliação é compensado por `groundedness` e `factual_consistency` determinísticos
Interface só por HTTP	Chamar o agente direto do Streamlit	Uma única implementação de agente e memória; a UI não precisa de Ollama nem de banco

7.5 Problemas encontrados e como foram resolvidos

7.5.1 Atualização do `langchain-core` exigida pelo `create_agent`

- **Sintoma:** o `langchain` 1.4.9 exige `langchain-core` 1.7.6; o projeto estava fixado em 1.6.12 desde a Fase 1.

- **Correção:** atualização do `langchain-core`.
- **Evidência:** a suíte offline das Fases 1 a 4 passou sem mudança depois da atualização (312 testes).

7.5.2 NLI com modelo de 2B: neutro classificado como contradição

- **Sintoma (1ª execução da integração, `docs/evidence/phase-5/integration-run1.log`):** "O Proof of Stake foi inventado em 2012 pela equipe do Peercoin", sem apoio nem negação na evidência, saiu como `contradiction`. O teste exigia acerto exato em 3 casos e falhou.
- **Causa:** o rótulo e a confiança são gerados pelo LLM, sem cabeça de classificação calibrada; afirmações "fora da evidência" tendem a ser lidas como "contra a evidência".
- **Correção:** a medição passou a usar 12 pares reais (4 por rótulo, sobre PoS e halving), com critério de pelo menos 10 acertos e no máximo 1 neutro classificado como contradição. A limitação foi documentada: a confiança não é probabilidade calibrada.

7.5.3 Pergunta sobre o próprio histórico sem resposta

- **Sintoma (mesma execução):** em "De qual ativo eu falei antes nesta conversa?", o agente respondeu que não havia histórico, embora as duas perguntas anteriores fossem sobre ETH. A verificação devolveu `verified` com 0 afirmações.
- **Leitura:** o histórico chegou ao agente (a pergunta seguinte, "volume de 24 horas desse mesmo ativo", resolveu ETH corretamente, inclusive após o reinício). O modelo de 2B não usou o histórico para responder uma pergunta **sobre** a conversa.
- **Limitação registrada:** a verificação confere afirmações factuais; uma recusa sem afirmações passa como `verified`. A avaliação mede a memória por outros meios (`memory-followup`, `memory-restart`).
- **Reprodução (integração final, 2026-11-08):** a mesma pergunta falhou de novo ("Não há nenhum ativo mencionado em sua solicitação anterior"). Desta vez a frase foi extraída como afirmação, ficou `neutral`, a verificação deu `unsupported`, a correção não resolveu e saiu a resposta cautelosa com aviso. A falha do modelo persiste; a diferença é que agora ela aparece como não verificada em vez de passar como `verified`.

7.5.4 Resposta cautelosa marcando a própria limitação

- **Sintoma (avaliação de base, `docs/evidence/phase-5/eval-local-baseline.json`):** em `ambiguous-01` e `error-coverage`, frases como "por favor, indique o ativo de seu interesse" receberam o sufixo "(não confirmado pelas fontes consultadas)".
- **Causa:** declarações de limitação, pedidos ao usuário e frases que só dizem de onde veio o dado eram extraídos como afirmações; sem evidência que os "sustentasse", ficavam `neutral` e a resposta cautelosa os marcava.
- **Efeito:** respostas corretas ficaram confusas, e a relevância medida caiu (`error-coverage`: 0,2).
- **Correção:** a extração passou a ignorar esses três tipos de frase (`NON_FACTUAL` e `PROVENANCE_ONLY` em `app/verification/claims.py`). Na avaliação final, os dois exemplos saem `verified` e sem marcação.

7.5.5 Avaliação contaminada por falhas de infraestrutura

- **Sintoma:** na execução `local-after-fixes` (`docs/evidence/phase-5/eval-local-after-fixes.json`), 8 dos 19 exemplos terminaram com `ConnectError('[Errno -2] Name or service not known')` ou `RemoteProtocolError`: o alvo perdeu o Ollama no meio da execução. As médias caíram (por exemplo, `tool_selection` de 0,929 para 0,5).
- **Causa:** o runner atribuía nota a exemplos que nem executaram; com saída vazia, os avaliadores davam 0.
- **Correção (2026-11-08):** `_summarize` exclui das médias os exemplos com erro e informa `n_scored` e `errored` (teste `test_summary_excludes_examples_that_failed_to_run`). A execução `local-after-fixes` não é usada como resultado; a avaliação final foi repetida (seção 6).

7.5.6 Interface: retomar conversa pelo id

- **Sintoma (2026-11-08, pedido do autor):** a barra lateral só permitia retomar uma conversa digitando o `session_id` (32 caracteres hexadecimais).
- **Correção:**
  - endpoint `GET /conversations` (mais recentes primeiro);
  - título gerado pelo modelo após o 1º turno, na manutenção em segundo plano (sem latência extra), com a primeira pergunta truncada como alternativa;
  - lista clicável na barra lateral, atualizada a cada 15 s;
  - botão “Deploy” do Streamlit oculto (`client.toolbarMode = "minimal"`).
- **Problema encontrado no teste real:** o título caiu sempre na alternativa. O LLM da aplicação roda em modo JSON (`format="json"`) e não devolve texto livre. O prompt passou a pedir `{"title": ...}`, e o parser aceita JSON ou texto. Na repetição, uma conversa sobre ETH recebeu “Preço e Consenso do Ethereum”.

7.5.7 Parâmetro depreciado na interface

- **Sintoma:** a auditoria final de depreciação encontrou `use_container_width` nos botões da interface, depreciado no Streamlit 1.65.
- **Correção:** `width="stretch"`. Detalhes da política no capítulo de decisões transversais.

7.6 Evidências e métricas

Verificação	Resultado
Spike de tool calling (`gemma4:e2b`, 2026-11-05)	`bind_tools` seleciona `get_price(BTC)` em 10,4 s; `create_agent` incorpora o resultado em 8,3 s; pergunta dupla chama as duas ferramentas; histórico resolve “meu favorito”
Integração da Fase 5, 1ª execução	5 passed, 1 failed (NLI, seção 5.2), 1 skipped (LangSmith não configurado), em 665,73 s
Avaliação de base (`local-baseline`, 19 exemplos, 2026-11-06)	Média de trajetória 0,927; média de resposta 0,869; 0 exemplos com erro
Avaliação final (`local-final`, 2026-11-08)	Ver a seção 6.1
Integração final da Fase 5 (2026-11-08)	6 passed, 1 skipped; ver a seção 6.2

Verificação	Resultado
Testes offline (Fases 1–5)	Ver o capítulo de decisões transversais (auditoria de depreciação)

7.6.1 Avaliação final

Execução local do dataset versão 2 (19 exemplos) em 2026-11-08, com o `gemma4:e2b` como agente, verificador e juiz (`docs/evidence/phase-5/eval-local-final.json`). Os **19 de 19 exemplos** executaram, sem erro de infraestrutura.

Média	Final	Base (2026-11-06)
Trajetória (7 avaliadores)	**0,958**	0,927
Resposta (6 avaliadores)	**0,978**	0,869

Avaliador	Final	Base
`groundedness`	1,000	1,000
`factual_consistency`	1,000	0,803
`logical_coherence`	1,000	1,000
`uncertainty`	1,000	0,929
`relevance`	0,986	0,857
`correctness`	0,879	0,686
`tool_selection`	0,929	0,929
`required_tools` / `no_unnecessary_tools` / `tool_combination` / `tool_inputs`	1,000	1,000
`evidence_used`	0,889	0,727
`error_handling`	0,750	0,750

Leitura:

- **Nenhum valor inventado:** `groundedness` 1,0 nas duas execuções; todo valor em US$ ou % das respostas existe nas observações reais.
- **Verificação:** os 14 exemplos do agente terminaram `verified` (na base: 8 `verified`, 3 `partially_verified`, 3 `unsupported`). Os 5 exemplos de verificação plantada (contradição, sem suporte, sustentada, cálculo errado e sentido invertido) receberam o status esperado nas duas execuções. Duas respostas passaram pela correção (`ambiguous-01` e `error-not-found`) e foram aprovadas na 2ª verificação.
- **Comparação com a base:** entre as duas execuções mudaram o código da verificação (seção 5.4) e o dataset (versão 1 → 2). A melhora não pode ser atribuída a uma só causa.
- **Falha que persiste:** em `error-not-found` (“preço da moeda XYZQWE”), o agente pediu esclarecimento sem consultar a API, em vez de buscar o ativo e informar que ele não existe (`tool_selection` 0, `error_handling` 0,5).
- **Viés do juiz:** em `memory-restart`, o juiz tirou nota porque “a data da fonte é futura”. O modelo julga 2026 pelo seu corte de treinamento, o mesmo efeito visto na Fase 1 (o modelo “corrigindo” o período do usuário). É mais um motivo para não depender só do juiz.
- **Latência:** de 14,0 s a 88,4 s por exemplo do agente, incluindo a verificação.

O resumo foi recalculado pelo `scripts/verify_report_numbers.py` a partir das linhas do arquivo, com a regra da seção 5.5, e coincide com o salvo.

7.6.2 Integração final

Execução de `tests/integration/test_phase5_live.py` com Ollama, CoinPaprika, SQLite, Chroma e Mem0 reais em 2026-11-08 (`docs/evidence/final/integration-phase5.log`): **6 passed, 1 skipped** em 403,56 s. O teste pulado é o de trace do LangSmith (variáveis não configuradas).

Teste	Resultado
Tool calling: bind, seleção, execução e incorporação	Aprovado
API, SQL, RAG e combinações (api+sql, api+rag)	Aprovado
Vários turnos, isolamento, reinício e Mem0	Aprovado (com a falha de conteúdo da seção 5.3 na pergunta sobre o próprio histórico)
NLI com 12 pares reais	Aprovado: acurácia 11/12; 4/4 `entailment`, 4/4 `contradiction`, 3/4 `neutral` (1 neutro classificado como contradição)
Correção única de resposta adulterada (preço de US$ 1.000.000)	Aprovado: o valor falso sai da resposta corrigida
Cliente Streamlit contra a API real	Aprovado

7.7 Limitações conhecidas

- **NLI por LLM de 2B:** confiança autodeclarada, sensível ao texto da evidência e lenta (10 a 40 s de verificação em respostas com várias afirmações conceituais).
- **Checagem numérica por presença:** confere se o número existe na evidência do mesmo ativo, não se foi atribuído à métrica certa.
- **Seleção de evidência lexical:** paráfrases sem palavras em comum podem ficar `neutral`.
- **Uma frase, uma afirmação:** uma frase com dois fatos vira uma afirmação só.
- **Recusas passam como `verified`** (seção 5.3).
- **LangSmith não exercitado neste ambiente:** o rastreamento e o runner estão implementados e testados offline, mas as variáveis `LANGSMITH_*` não foram configuradas; o teste de trace foi pulado e os resultados são da execução local equivalente.
- **Juiz com viés de autoavaliação:** o mesmo modelo responde e julga; as notas do juiz são indício, não prova.
- **Latência:** um turno leva de 14 s a quase 3 min em CPU; cada correção é mais uma execução do agente.

7.8 Aprendizados

- **Verificar exige separar tipos de afirmação.** Números por código, conceitos por NLI, coerência por regras e só o que sobra pelo LLM. Um verificador único por LLM herdaria os mesmos erros do agente.

- **“Sem evidência” e “contra a evidência” são coisas diferentes,** e o modelo pequeno confunde as duas. O desenho de status (`neutral`, `unsupported`, `verification_failed`) existe para não punir a resposta pela limitação da verificação.
- **Avaliação precisa distinguir falha do sistema de falha da infraestrutura.** Médias com exemplos que nem rodaram contam uma história falsa.
- **Depreciação continua aparecendo no fim:** até a interface tinha um parâmetro depreciado. A auditoria com todos os avisos visíveis deve fazer parte da entrega.
- **Modo JSON é configuração global do LLM.** Uma tarefa nova de texto livre (o título) quebrou em silêncio; o teste com o modelo real revelou o que o dublê não revelaria.

7.9 Pendências

- Configurar o LangSmith e repetir a avaliação como experimento rastreado.
- Extração de afirmações com divisão de fatos compostos e checagem de atribuição de métrica.
- Não marcar declarações de limitação na resposta cautelosa.
- NLI com cabeça treinada (Transformers) se houver disco, mantendo a mesma interface `classify`.
- Autenticação e dono por sessão antes de qualquer uso com mais de um usuário.

8 Decisões transversais

Este capítulo reúne as decisões que valem para o projeto inteiro. Os detalhes de cada uma estão nos capítulos das fases e nos documentos de `docs/`.

8.1 Política de não usar APIs depreciadas

8.1.1 A diretriz

A partir da Fase 3, o projeto adotou uma regra explícita: **nenhum código novo usa classe ou função marcada como depreciada**, mesmo quando o enunciado da fase a pede pelo nome. Quando um spike ou uma checagem encontrava um aviso de depreciação, a implementação parava e o caso era decidido antes de seguir. Em todos os casos a decisão foi trocar pela API atual equivalente e registrar o desvio.

Os motivos:

1. **Prazo de remoção conhecido.** Os avisos encontrados trazem a versão de remoção (LangChain 2.0, LangGraph 2.0). Código novo com essas APIs nasceria com data de expiração.
2. **A própria biblioteca indica o substituto.** Os avisos apontam para `create_agent` e para a persistência do LangGraph. Seguir a recomendação mantém o projeto no caminho que a biblioteca vai continuar mantendo.
3. **Avisos escondem problemas.** Uma execução que imprime avisos em todo import treina a equipe a ignorá-los. Com zero avisos, qualquer aviso novo é sinal de que algo mudou.

8.1.2 O que foi trocado

Fase	Pedido pelo enunciado ou candidato	Situação verificada	Substituto adotado
2	FAISS via `langchain-community`	Pacote em descontinuação (“is being sunset”), sem `langchain-faiss` no PyPI; índice persistido com pickle, exigindo desserialização insegura	Chroma embutido via `langchain-chroma`, pacote dedicado
3	`ConversationBufferMemory`	Depreciado desde o LangChain 0.3.1, remoção na 2.0	Histórico próprio em SQLite + janela e resumo incremental próprios
3	`InMemoryChatMessageHistory`	Depreciado no `langchain-core` 1.6.4, remoção na 2.0	O mesmo histórico em SQLite

Fase	Pedido pelo enunciado ou candidato	Situação verificada	Substituto adotado
3	LangMem (framework de memória)	A dependência obrigatória `trustcall` importa `Send` de `langgraph.constants`, depreciado no LangGraph 1.0; o aviso aparece em qualquer uso	Mem0 2.2.1, sem nenhum aviso nos spikes
5	`create_openai_tools_agent` + `AgentExecutor`	Sem aviso, mas pertencem ao pacote legado `langchain-classic`	`create_agent` do pacote `langchain` 1.4.9 (grafo LangGraph)
5	`RunnableWithMessageHistory`	Aviso no `langchain-core` 1.7.6: "Use LangGraph's built-in persistence instead"; `BaseChatMessageHistory` também depreciado	`SessionHistoryAdapter`, que lê o SQLite e o Mem0 da Fase 3
5	Streamlit `use_container_width`	Depreciado no Streamlit 1.65 ("will be removed in a future release")	`width="stretch"`

O caso do LangMem mostrou que a depreciação também vem de **dependências transitivas**: o pacote em si não usa nada depreciado, mas uma dependência obrigatória usa. Por isso a checagem foi feita com a pilha real instalada, e não só pela documentação.

8.1.3 Como a regra foi verificada

- **Spikes com avisos visíveis:** cada candidato foi importado e executado com `warnings.simplefilter("always")` no ambiente da imagem antes da adoção.
- **Teste automático:** `test_phase3_modules_use_no_deprecated_langchain_apis` importa os módulos da Fase 3 num interpretador limpo, com `LangChainDeprecationWarning` tratado como erro. Rodar num subprocesso foi necessário porque recarregar módulos no mesmo processo recriava classes de exceção e quebrava outros testes.
- **Auditoria final (2026-11-08):** a suíte offline completa rodou com `-W always::DeprecationWarning -W always::PendingDeprecationWarning -W always::FutureWarning`: **370 testes aprovados e nenhum aviso** (`docs/evidence/final/pytest-deprecation-audit.log`).

8.1.4 A exceção: APIs legadas exigidas na Fase 2

O enunciado da Fase 2 exigia `initialize_agent`, `ZERO_SHOT_REACT_DESCRIPTION` e `RouterChain`. No LangChain 1.x essas APIs saíram do pacote `langchain` e vivem no `langchain-classic`. A política ainda não existia, e elas foram usadas como pedido, com duas salvaguardas:

- ficam **isoladas** em `app/legacy/`, consumidas por interfaces modernas (o `CryptoAgent` cuida de memória, rastreamento e montagem da saída; o `RouterChain`

só classifica);
- atendem só os endpoints da Fase 2 (`POST /agent` e `POST /route?router=legacy`).

Na Fase 5 o agente principal migrou para `create_agent`. O código legado continua no repositório só para preservar a Fase 2, e removê-lo afeta apenas `app/legacy/`.

8.2 Desvios deliberados do enunciado

Todos foram decididos pelo autor, com o motivo registrado na fase correspondente.

Fase	Enunciado	Implementado	Motivo
2	Scripts Python	Aplicação HTTP (FastAPI)	Uma entrada única com dependências criadas uma vez, no lugar de oito scripts com inicialização repetida
2	FAISS	Chroma	Descontinuação do `langchain-community`, pickle inseguro, sem atualização incremental nem filtro por metadados
3	`ConversationBufferMemory`	SQLite + janela + resumo próprios	Depreciado
3	Só comparar frameworks de memória	Usar um (Mem0)	Pedido do autor; LangMem descartado por depreciação transitiva, Letta por imagem Docker sem suporte
5	`create_openai_tools_agent` + `AgentExecutor`	`create_agent`	Pacote legado; `create_agent` é o substituto oficial
5	`RunnableWithMessageHistory`	`SessionHistoryAdapter`	Depreciado
5	NLI com Transformers (`mDeBERTa-v3-base-mnli-xnli`)	NLI com `gemma4:e2b` via Ollama	Cerca de 2 GB a mais (torch, transformers e modelo) com 5 GB livres em disco, e um segundo motor de inferência na mesma CPU

8.3 Modelo local (Ollama) em CPU

8.3.1 Por que um modelo local e pequeno

A escolha de servir o modelo localmente pelo Ollama, em vez de usar uma API comercial, partiu da natureza do problema:

1. **O conhecimento que importa vem das ferramentas.** Uma análise de mercado cripto depende de preços, volumes e capitalizações que mudam a cada minuto. Nenhum modelo, por maior que seja, tem esses valores no treinamento. A qualidade da resposta depende de o agente **escolher a ferramenta certa, passar os argumentos certos e usar o resultado sem inventar**, e não da

memória do modelo. Por isso um modelo pequeno é suficiente, desde que siga o protocolo de ferramentas. O projeto serviu justamente para medir esse potencial: até onde um modelo local leve consegue orquestrar ferramentas reais.
2. **Privacidade.** Perguntas, histórico de conversa e memórias de longo prazo ficam na máquina. Só as consultas à CoinPaprika (símbolos de ativos) saem para a internet; o LangSmith é opcional e desligado por padrão.
3. **Avaliar o estado dos modelos locais.** Outro objetivo era verificar a qualidade que modelos abertos já alcançam num computador pessoal, sem GPU dedicada.
4. **Custo e reprodutibilidade.** Sem cobrança por token e sem dependência de disponibilidade de um provedor; qualquer pessoa reproduz o ambiente com `make up`.

8.3.2 Hardware

Todas as medições foram feitas num notebook com **AMD Ryzen 5 5500U** (6 núcleos, 12 threads), 22 GB de RAM e **sem GPU dedicada**. A inferência roda só na CPU.

8.3.3 Escolha do modelo: benchmark

O benchmark da Fase 1 (`benchmarks/results/`) comparou cinco configurações nas mesmas 6 perguntas, medindo latência, validade do JSON e acerto da classificação:

Modelo	Thinking	Latência média (s)	JSON válido	Classificação correta
`qwen2.5:7b`	—	71,0	100%	83%
`gemma4:e4b`	desligado	83,1	100%	100%
`gemma4:e4b`	ligado	118,1	100%	100%
`mistral:7b`	—	141,9	67%	100%[1]
`gemma4:e2b`	desligado	**42,3**	100%	100%

[1] Só sobre as respostas válidas.

O **`gemma4:e2b` (família Gemma 4) com *thinking* desligado** foi o mais rápido (cerca de 40% menos latência que o `qwen2.5:7b`), com JSON sempre válido e acerto máximo nas perguntas do conjunto. Ele virou o padrão em 2026-10-18 e foi mantido até o fim. Na Fase 5, o mesmo modelo provou suportar tool calling nativo pelo Ollama, o que permitiu migrar o agente sem trocar de modelo.

8.3.4 Resultado

Considerando o hardware, o desempenho foi razoável: na avaliação final da Fase 5, o modelo escolheu a categoria de ferramenta certa em 13 dos 14 exemplos do agente (`tool_selection` 0,929), combinou API, SQL e documentos na mesma resposta e não inventou valores de mercado (`groundedness` 1,0 nas avaliações da Fase 5). Um turno completo do agente integrado, com verificação, levou de 14 s a 88 s na avaliação final; respostas com correção ou várias ferramentas chegaram a quase 3 min em execuções anteriores.

O preço de um modelo de 2B em CPU aparece em todas as fases: latência de dezenas de segundos por chamada e erros de raciocínio numérico. As duas decisões seguintes são a resposta a isso.

8.4 Código determinístico onde o modelo pequeno falha

Uma lição repetida em todas as fases: **o que pode ser calculado por código não deve ser pedido ao modelo**.

Fase	O modelo errou	Passou a ser feito por código
1	Omitia a etapa de coleta de dados	Etapa derivada de `requested_metrics` e registrada em `assumptions`
1	Escrevia valores de mercado sem fonte	Trava que remove frases com valores e acrescenta aviso
2	Invertia comparações, mesmo com o ranking no prompt	Conclusão da rota de comparação montada por código
2–4	Inventava fontes e horários	Achados, fontes e horários extraídos das observações (grounding)
3	Instável para detectar período, correção e ativos	Regras determinísticas prevalecem sobre o LLM na intenção
4	Errava JOIN e copiava filtros indevidos	View `snapshot_history`, período por regra, SQL validado e parametrizado
5	Comparações numéricas na verificação	Checagem numérica determinística; o LLM só julga afirmações conceituais

8.5 Segurança

- **SQL gerado por LLM** passa por duas camadas: parser (`sqlglot`) antes da execução e conexão somente leitura com `set_authorizer` e `progress_handler` durante. Os literais viram parâmetros.
- **Sem pickle:** a troca para o Chroma eliminou o `allow_dangerous_deserialization`.
- **Segredos:** chaves ficam no `.env` (fora do git); os logs redigem a chave da CoinPaprika; os traces do LangSmith recebem só um hash do `session_id`.
- **Sem autenticação:** o `session_id` não é credencial, e a listagem de conversas expõe todos os ids. Isso é aceitável apenas no uso local de um único usuário e está registrado como limitação.

8.6 Estratégia de testes

- **Testes offline** (padrão do `pytest`): LLM falso, API simulada e bancos temporários. Rodam em segundos e cobrem contratos, regras determinísticas, segurança do SQL e fluxos completos da API.
- **Testes de integração** (opt-in com `-m integration` e `-m network`): Ollama, CoinPaprika, Chroma e Mem0 reais. Como a saída do modelo varia mesmo com temperatura 0, esses testes usam critérios de comportamento (ferramenta certa, melhora de ranking, status esperado), e não igualdade de texto.
- **Avaliação (Fase 5):** dataset versionado de 19 exemplos, avaliadores de resposta e de trajetória, execução local ou no LangSmith.

9 Conclusão

9.1 O que foi entregue

Ao fim das cinco fases, o Crypto Intelligence Agent é uma aplicação local completa:

- uma API HTTP que preserva cada fase (plano estruturado, agente ReAct, chains roteadas, conversa com memória, Text-to-Query, Retrieval Agent e agente integrado);
- um agente com tool calling nativo que combina dados ao vivo, snapshots em SQLite, documentos no Chroma e cálculo determinístico na mesma resposta;
- memória persistente por sessão (SQLite como fonte da verdade, Mem0 para memória episódica e semântica), que sobrevive a reinícios;
- verificação de cada resposta contra a evidência da própria execução, com status explícito e no máximo uma correção;
- avaliação reproduzível com dataset versionado e avaliadores de resposta e de trajetória;
- uma interface Streamlit com lista de conversas tituladas pelo modelo e detalhes da verificação em cada resposta.

9.2 Resultados consolidados

Fase	Medição	Resultado
1	Benchmark de modelos em CPU (6 perguntas)	`gemma4:e2b` sem thinking: 42,3 s de latência média, 100% de JSON válido
2	Agente ReAct com dados reais	Valores, fontes e horários reais; comparação de 3 ativos com conclusão correta montada por código
3	Integração com Ollama, Mem0 e CoinPaprika reais	Continuação, correção, resultado anterior, reinício e isolamento entre sessões corretos
4	Text-to-Query real (9 perguntas)	9 status corretos, nenhum SQL inseguro executado
4	Diagnóstico de recuperação (halving)	Trecho relevante do 3º para o 1º lugar com Query Rewriting (MRR 0,33 → 1,0)
5	Avaliação final do agente integrado	19 de 19 exemplos executados; média de trajetória 0,958 e de resposta 0,978; `groundedness` 1,0; os 14 exemplos do agente terminaram `verified`
1–5	Suíte offline com todos os avisos visíveis	370 testes aprovados, nenhum aviso de depreciação

9.3 O que funcionou

- **Ferramentas no lugar de memória do modelo.** Com os dados vindo das ferramentas, um modelo de 2B em CPU foi suficiente para orquestrar API, SQL e documentos. A hipótese que motivou o uso de um modelo local pequeno se confirmou.
- **Código determinístico nos pontos fracos do modelo.** Comparações, rankings, períodos, checagem de números e montagem de fontes saíram do LLM e passaram a ser feitos por código. Isso eliminou classes inteiras de erro observadas nas primeiras fases.

- **Separar "sem evidência" de "contra a evidência".** O desenho de status da verificação permite entregar respostas parcialmente sustentadas com transparência, em vez de rejeitá-las ou aprová-las por inteiro.
- **APIs atuais desde o início de cada fase.** A política de não usar nada depreciado evitou retrabalho: a migração do agente para `create_agent` na Fase 5 só tocou `app/integrated/`, e a auditoria final terminou sem avisos.

9.4 Limitações

- **Latência em CPU:** um turno completo leva de 14 s a quase 3 min.
- **Verificação com modelo de 2B:** a NLI tem confiança autodeclarada e confunde "sem evidência" com "contradição" em parte dos casos; a checagem numérica confere presença de valor, não a métrica a que ele foi atribuído.
- **Juiz e agente são o mesmo modelo:** as notas do juiz são indício, compensado por avaliadores determinísticos.
- **Dados:** o plano gratuito da CoinPaprika não oferece série histórica; o histórico local depende de coletas manuais.
- **Sem autenticação:** adequado só ao uso local de um usuário.
- **LangSmith:** implementado, mas não exercitado neste ambiente; os resultados vêm da execução local equivalente.

9.5 Trabalhos futuros

1. NLI com modelo de classificação dedicado (mesma interface `classify`), se houver disco disponível.
2. Coleta agendada de snapshots e uma fonte de séries históricas (CoinGecko) para perguntas de longo prazo.
3. Extração de afirmações com divisão de fatos compostos e checagem da métrica atribuída a cada número.
4. Remoção do código legado da Fase 2 (`app/legacy/`), já substituído pelo agente integrado.
5. Autenticação e dono por sessão antes de qualquer uso multiusuário.
6. Experimentos no LangSmith para comparar versões de prompt e de modelo ao longo do tempo.

Referências

Artigos

- LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020.
- MA, X. et al. Query Rewriting for Retrieval-Augmented Large Language Models. *EMNLP*, 2023.
- NUSSBAUM, Z. et al. Nomic Embed: Training a Reproducible Long Context Text Embedder. *arXiv:2402.01613*, 2024.
- WILLIAMS, A.; NANGIA, N.; BOWMAN, S. A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. *NAACL*, 2018.
- YAO, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023.

Modelos

- Google DeepMind. Gemma 4 (`gemma4:e2b`, `gemma4:e4b`), via biblioteca de modelos do Ollama: https://ollama.com/library/gemma4.
- Qwen Team. Qwen2.5 (`qwen2.5:7b`): https://ollama.com/library/qwen2.5.
- Mistral AI. Mistral 7B (`mistral:7b`): https://ollama.com/library/mistral.
- Nomic AI. `nomic-embed-text`: https://ollama.com/library/nomic-embed-text.
- LAURER, M. `MoritzLaurer/mDeBERTa-v3-base-mnli-xnli` (sugerido pelo enunciado, não adotado): https://huggingface.co/MoritzLaurer/mDeBERTa-v3-base-mnli-xnli.

Fonte de dados

- CoinPaprika API: https://api.coinpaprika.com/.

Bibliotecas e documentação

- LangChain (`langchain`, `langchain-core`, `langchain-classic`, `langchain-ollama`, `langchain-chroma`): https://docs.langchain.com/.
- LangGraph: https://docs.langchain.com/oss/python/langgraph/overview.
- LangSmith: https://docs.smith.langchain.com/.
- Ollama: https://ollama.com/.
- Chroma: https://docs.trychroma.com/.
- Mem0: https://docs.mem0.ai/.
- sqlglot: https://github.com/tobymao/sqlglot.
- SQLite (`sqlite3.Connection.set_authorizer`, modo somente leitura por URI): https://docs.python.org/3/library/sqlite3.html.
- FastAPI: https://fastapi.tiangolo.com/.
- Pydantic: https://docs.pydantic.dev/.
- Streamlit: https://docs.streamlit.io/.